

TechTribers

Java Interview Series · Part 23 · Q1149–Q1208

Java Performance & JVM Configuration

A measurement-first method, heap and native-memory sizing, GC tuning recipes, production JVM flags and container settings, TLS, DNS and encoding configuration, JIT and startup time, code-level performance, and runbooks for leaks, high CPU and latency spikes.

60	60	6	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `PF-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**
A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**
What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**
The next question the interviewer is likely to ask, with the answer.

60	7	60	6
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

📏	01	Measuring and Profiling Java Performance	Q1149–Q1158	5
🧮	02	Heap and Memory Configuration	Q1159–Q1168	13
♻️	03	Garbage Collection Tuning in Practice	Q1169–Q1177	21
🎛️	04	JVM Flags, System Properties and Runtime Configuration	Q1178–Q1188	29
🔥	05	JIT, Startup Time and Warm-up	Q1189–Q1194	38
⚡	06	Code-Level Performance	Q1195–Q1203	44
🚨	07	Production Diagnostics and Runbooks	Q1204–Q1208	53

© TechTribers. For personal use only. Do not redistribute.

All questions

📏 Measuring and Profiling Java Performance

Q1149 Walk through diagnosing a slow production Java service, step by step. 5

Q1150 Latency vs throughput vs tail latency: which percentiles matter, and what does Little's law say about concurrency? 6

Q1151 How do you write a correct microbenchmark with JMH, and what pitfalls does it protect you from? 7

Q1152 Why does the first request take so long, and why do naive benchmarks lie about the JVM? 8

Q1153 Sampling vs instrumenting profilers: JFR, async-profiler and safepoint bias. 8

Q1154 How do you read a flame graph, and what is the difference between CPU, wall-clock, allocation and lock profiles? 9

Q1155 Which jcmd, jstat, jmap and jinfo commands should you know for inspecting a running JVM? 10

Q1156 How do you profile continuously in production with an overhead budget? 11

Q1157 How do you load-test a JVM service correctly (warm-up, steady state, coordinated omission)? 11

Q1158 How do you find lock contention and thread starvation in a Java service? 12

🧮 Heap and Memory Configuration

Q1159 How do you size the Java heap (-Xms, -Xmx) for a service? 13

Q1160 Should -Xms equal -Xmx, and what does -XX:+AlwaysPreTouch do? 14

Q1161 Why does a JVM process use more memory than -Xmx? (the full footprint) 15

Q1162 How do you use Native Memory Tracking (NMT) to find where non-heap memory goes? 16

Q1163 How do you size and protect Metaspace (MaxMetaspaceSize, class-loader leaks)? 16

Q1164 How does thread count affect memory, and when do you tune -Xss? 17

Q1165 How do you size and monitor direct (off-heap) memory, and what happens when it runs out? 18

Q1166 What are compressed oops and compressed class pointers, and why does crossing 32 GB of heap cost memory? 19

Q1167 When do huge pages (Transparent Huge Pages, -XX:+UseLargePages) help a JVM, and what are the risks? 19

Q1168 How do you reduce the memory footprint of a Java service? 20

♻️ Garbage Collection Tuning in Practice

Q1169 Which G1 tuning flags matter (MaxGCPauseMillis, region size, IHOP), and which should you leave alone? 21

Q1170 What are humongous objects in G1, why do they hurt, and how do you fix them? 22

Q1171 Diagnose these GC symptoms: frequent young GCs, long pauses, Full GC, to-space exhausted, allocation stalls. 23

Q1172 How do allocation rate and premature promotion affect GC, and how do you reduce them? 24

Q1173 When is Parallel GC a better choice than G1 (batch and throughput workloads)? 24

Q1174 How do you tune ZGC (SoftMaxHeapSize, heap headroom, allocation stalls, uncommit)? 25

Q1175 ZGC vs Shenandoah: how do you choose between the low-pause collectors? 26

Q1176 How do container CPU limits affect GC thread counts and the choice of collector? 27

Q1177 How do you reduce GC pressure in code? (pooling myths, hot-path allocations, escape analysis) 27

© TechTribers. For personal use only. Do not redistribute.

🎚️ JVM Flags, System Properties and Runtime Configuration

Q1178 What kinds of JVM options exist, and where can you set them (command line, JAVA_TOOL_OPTIONS, JDK_JAVA_OPTIONS, argument files)? 29

Q1179 Give a production-ready JVM options baseline for a Spring Boot service on Kubernetes, with the reasoning for each flag. 30

Q1180 How should you set Kubernetes requests and limits for a JVM workload (memory, CPU, QoS, throttling)? 31

Q1181 Which system properties (-D) matter for a Java service: encoding, temp dir, IPv4, headless, JMX? 32

Q1182 How do you configure TLS and truststores for the JVM (custom CA, javax.net.ssl, protocols)? 32

Q1183 How does JVM DNS caching work, and how do you configure it (networkaddress.cache.ttl, keep-alive, timeouts)? 33

Q1184 How do entropy and SecureRandom affect startup and performance (securerandom.source, egd)? 34

Q1185 Which old JVM flags no longer work, and what replaced them (PermSize, CMS, biased locking, PrintGC)? 35

Q1186 What do you need to know about default charset, time zone and locale on the JVM? 36

Q1187 What are --add-opens, --add-exports, --enable-preview and --enable-native-access, and where do you set them? 36

Q1188 How should you launch the JVM in Docker (exec form, PID 1, signals, JAVA_OPTS, graceful shutdown)? 37

🔥 JIT, Startup Time and Warm-up

Q1189 How does tiered compilation work, and what is the code cache (ReservedCodeCacheSize)? 38

Q1190 What defeats JIT inlining and makes hot code slow (megamorphic calls, huge methods, exceptions, reflection)? 39

Q1191 How do you cut JVM startup time (AppCDS, JDK AOT cache, lazy initialisation)? 40

Q1192 What are CRaC and checkpoint/restore, and how do they compare with the AOT cache and GraalVM native image? 41

Q1193 How do you warm up JVM pods in Kubernetes so users don't see slow first requests? 42

Q1194 Give a startup and memory checklist for a Spring Boot service. 43

⚡ Code-Level Performance

Q1195 How does collection resizing cost performance, and how do you presize ArrayList, HashMap and StringBuilder? 44

Q1196 When are streams slower than loops, and when does parallel() help or hurt? 45

Q1197 What does synchronization really cost, and when do LongAdder, StampedLock and @Contended help? 46

Q1198 How do you size thread pools (CPU-bound vs I/O-bound, queues, ForkJoin, virtual-thread scheduler)? 47

Q1199 How do you use an in-process cache like Caffeine correctly (size, expiry, stampede, memory)? 48

Q1200 How do you reduce JSON serialization cost with Jackson (ObjectMapper reuse, streaming, Blackbird, payload size)? 49

Q1201 Which I/O patterns matter for performance (buffering, NIO, zero-copy, memory mapping, HTTP client reuse)? 50

Q1202 How much does logging cost, and how do you keep it from hurting latency (async appenders, parameterised messages)? 51

Q1203 What are the regex performance pitfalls (recompiling patterns, catastrophic backtracking / ReDoS)? 52

🚨 Production Diagnostics and Runbooks

Q1204 What are the most common memory-leak patterns in Java and Spring services, and how do you spot each? 53

Q1205 How do you diagnose high CPU usage in a Java process? 54

Q1206 The service is up but not responding. What do you check, in order? 55

Q1207 Latency spikes every few minutes: how do you find the cause? 55

Q1208 Which JVM metrics and alerts should every service have (Micrometer, Prometheus)? 56

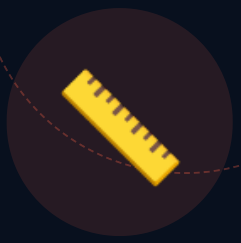
© TechTribers. For personal use only. Do not redistribute.

Section 01

Measuring and Profiling Java Performance

A diagnosis runbook, latency versus throughput and tail percentiles, correct microbenchmarks with JMH, why warm-up fools naive benchmarks, sampling profilers and flame graphs, the jcmd toolbox, continuous profiling, and load testing without coordinated omission.

Questions 1149–1158 · 10 questions



Q1149 PF-001

• Advanced

Walk through diagnosing a slow production Java service, step by step.

⚡ Quick answer

Work in a loop and **never tune before measuring**. **1. Define the symptom** (which endpoint, which percentile, since when, what changed). **2. Find the saturated resource** with the USE method (utilisation, saturation, errors) for CPU, memory, disk, network and **pools** (threads, connections). **3. Locate it in the process**: CPU profile, GC log, thread dump, lock profile, or traces across services. **4. Form one hypothesis**, change one thing, **5. re-measure** under the same load, and keep the result only if the numbers moved.

The expensive mistakes are all about **skipping steps**: tuning GC when the problem is a slow downstream call, enlarging a pool when the problem is lock contention, or declaring victory after one benchmark run on a cold JVM. Write down hypothesis, change and measured effect: that record is what makes the next incident faster.

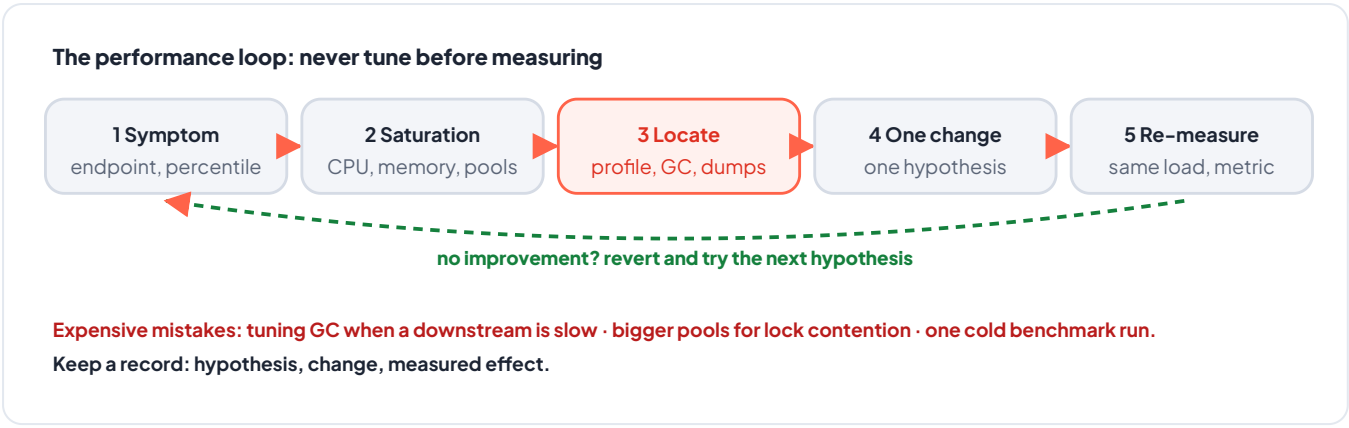
Terminal

```
# 1-2. Is it the host, the JVM, or something it waits on?
top -H -p <pid>                      # per-thread CPU: one hot thread or all busy?
vmstat 1 | head; iostat -x 1 3       # run queue, swapping, disk saturation
ss -s                                # socket counts, retransmits (network side)

# 3. Inside the JVM
jstat -gcutil <pid> 1s               # GC share of time, old-gen occupancy
jcmd <pid> Thread.print > t1.txt     # who is RUNNABLE, BLOCKED, WAITING (take 3 dumps, 5 s apart)
jcmd <pid> JFR.start duration=60s filename=/tmp/slow.jfr settings=profile
asprof -d 30 -e cpu -f /tmp/cpu.html <pid>      # CPU flame graph (async-profiler)

# 4-5. One change, same load, compare p50/p95/p99 and throughput, not a single run
```


© TechTribers. For personal use only. Do not redistribute.



📈 **Q1150** PF-002 • Intermediate

Latency vs throughput vs tail latency: which percentiles matter, and what does Little's law say about concurrency?

⚡ **Quick answer**

Throughput is work per second; **latency** is time per request. Averages hide problems, so use **percentiles** (p50, p95, p99, p99.9): a service with a 40 ms mean can have a 2 s p99, and users who hit many requests per page **experience the tail**. **Little's law**: *concurrency = throughput × latency*; 2,000 requests/s at 50 ms means 100 requests in flight, which sizes pools and explains why a slow dependency exhausts threads fast.

Define **SLOs on percentiles** (for example p99 under 300 ms) and watch the **error budget**. Percentiles can't be averaged across instances: aggregate histograms (buckets) and compute the percentile afterwards, as Prometheus does with `histogram_quantile`.

Java

```
// Little's law as a sizing check
double throughput = 2_000;        // requests per second
double latencySeconds = 0.050;    // average time a request holds a worker
double inFlight = throughput * latencySeconds;          // = 100 concurrent requests

// If the downstream gets slow: same arrival rate, 10x latency
double inFlightSlow = throughput * 0.500;               // = 1,000: a 200-thread pool saturates

// Record latency with a histogram, never a running average (Micrometer)
Timer timer = Timer.builder("orders.create")
        .publishPercentiles(0.5, 0.95, 0.99)
        .publishPercentileHistogram()
        .register(meterRegistry);
timer.record(() -> orderService.create(request));
```


© TechTribers. For personal use only. Do not redistribute.

🧪 **Q1151** PF-003 • Advanced

How do you write a correct microbenchmark with JMH, and what pitfalls does it protect you from?

⚡ **Quick answer**

Hand-written loops with `System.nanoTime()` mislead because the JIT **eliminates dead code**, **constant-folds** inputs, **inlines** differently than in real code, and needs **warm-up**. **JMH** (the OpenJDK harness) handles warm-up, forks a fresh JVM per trial, controls inlining, and offers `Blackhole` to consume results so the work can't be optimised away. Benchmark the **hot method in isolation only to compare alternatives**, then confirm in the real application.

Java

```
@State(Scope.Benchmark)
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
@Warmup(iterations = 5, time = 1)
@Measurement(iterations = 5, time = 1)
@Fork(2)                                        // separate JVMs: avoids profile pollution
public class ConcatBenchmark {

    @Param({"10", "1000"})
    int size;
    String[] parts;

    @Setup
    public void setup() {
        parts = IntStream.range(0, size).mapToObj(i -> "item" + i).toArray(String[]::new);
    }

    @Benchmark
    public String stringBuilder() {
        StringBuilder sb = new StringBuilder();
        for (String p : parts) sb.append(p).append(',');
        return sb.toString();                   // returned value is consumed by JMH
    }

    @Benchmark
    public void joiner(Blackhole bh) {
        bh.consume(String.join(",", parts));    // Blackhole prevents dead-code elimination
    }
}
// java -jar target/benchmarks.jar -prof gc     (also reports allocation per operation)
```

⚠️ **Interview trap**

Benchmarking with constant inputs (`compute(42)`) in a loop whose result is unused: the JIT folds or deletes the work and you measure nothing. Use `@State` fields and return or `Blackhole.consume` every result.

© TechTribers. For personal use only. Do not redistribute.

Q1152 PF-004

• Intermediate

Why does the first request take so long, and why do naive benchmarks lie about the JVM?

⚡ Quick answer

Code starts **interpreted**, is compiled by **C1** after a few hundred invocations, and by **C2** (heavily optimised) after thousands, based on **runtime profiles**. Classes load lazily, caches are cold and the JIT competes with your app for CPU. So the first seconds are **slower by 10–100×** than steady state, and a benchmark that is too short, or whose workload changes shape, measures warm-up and deoptimisation instead of performance. **Deoptimisation** happens when an assumption (a monomorphic call site, a type) breaks.

Consequences for engineering:

- **Warm up before measuring** and before taking traffic (Q1193), and run load tests long enough to reach steady state.
- A new code path (an error branch, a rarely used endpoint) is **cold** and slow the first time even in a warm service.
- **CPU throttling** in containers stretches warm-up dramatically because compiler threads compete with request threads.

Terminal

```
# Watch compilation happen
java -XX:+PrintCompilation -jar app.jar | head -40
#   timestamp  id  flags  tier  method                          size
#     312   45     n   0   java.lang.Object::hashCode (native)
#     340  120       3       com.acme.PriceCalc::total (28 bytes)     <- tier 3: C1 with profiling
#     611  121       4       com.acme.PriceCalc::total (28 bytes)     <- tier 4: C2, fully optimised
#     902  120   made not entrant  com.acme.PriceCalc::total        <- deoptimised: assumption broke

# Tiered compilation thresholds and compiler threads are adaptive; see the JIT section for
#   tuning
```

Q1153 PF-005

• Advanced

Sampling vs instrumenting profilers: JFR, async-profiler and safepoint bias.

⚡ Quick answer

An **instrumenting** profiler rewrites methods to record entry/exit: exact counts, but heavy overhead that distorts results. A **sampling** profiler periodically captures stack traces (here 10–100 times a second) and is cheap enough for production. Many older sampling profilers could only sample at **safepoints**, which biases results toward loop back-edges and method returns (**safepoint bias**). **Java Flight Recorder** (built into the JVM) and **async-profiler** (uses `AsyncGetCallTrace` /perf events) sample **without** needing safepoints, and async-profiler also does **wall-clock**, **allocation** and **lock** profiling.

© TechTribers. For personal use only. Do not redistribute.

JFR is the right first tool because it ships with the JDK and is safe in production (Q336). Reach for async-profiler when you need **native frames**, wall-clock views or the best flame graphs. In containers async-profiler may need `--cap-add SYS_ADMIN` or `-e itimer` when `perf_events` isn't available.

Terminal

```
# JFR: built in, low overhead, great default
java -XX:StartFlightRecording=filename=/tmp/app.jfr,settings=profile,duration=120s -jar app.jar
jcmd <pid> JFR.start name=slow settings=profile duration=60s filename=/tmp/slow.jfr
jfr print --events jdk.ExecutionSample /tmp/slow.jfr | head          # CLI inspection
jfr summary /tmp/slow.jfr                                            # what was recorded

# async-profiler: flame graphs, wall clock, allocation, locks
asprof -d 30 -e cpu   -f cpu.html   <pid>       # where CPU time goes
asprof -d 30 -e wall  -f wall.html  <pid>       # where threads wait (I/O, locks, sleeps)
asprof -d 30 -e alloc -f alloc.html <pid>       # who allocates the most
asprof -d 30 -e lock  -f lock.html  <pid>       # lock contention
```

Q1154 PF-006 • Intermediate

How do you read a flame graph, and what is the difference between CPU, wall-clock, allocation and lock profiles?

⚡ **Quick answer**

In a flame graph each box is a **method**, **width is the share of samples** (time or allocations), vertical position is **call depth** (callers below, callees above), and the colour is arbitrary. Look for **wide plateaus at the top**: that's where time is actually spent. A **CPU** profile shows only on-CPU work; a **wall-clock** profile includes waiting (I/O, locks, sleep), so it explains latency; an **allocation** profile shows who creates garbage; a **lock** profile shows contention.

Terminal

```
# Read it like this
#   1. Find the widest boxes near the TOP edge (self time), not the base.
#   2. Walk down: who calls it? Is the caller expected to do this much work?
#   3. Compare against a baseline: differential flame graph (before vs after a change)

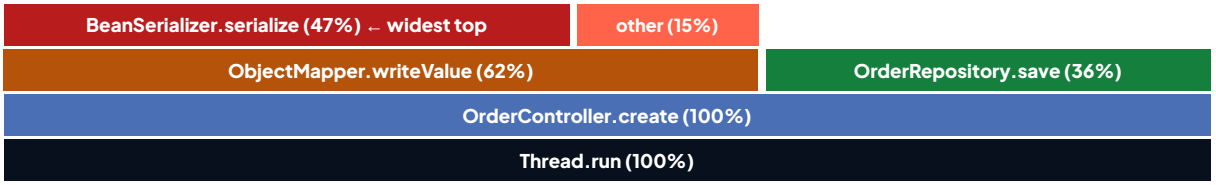
# Typical findings, by profile type
#   cpu   : JSON serialisation 25%, regex 12%, String.format 8%      -> optimise the hot code
#   wall  : 70% in SocketInputStream.read under HttpClient.send       -> waiting on a downstream
#   call
#   alloc : 40% of bytes allocated in Arrays.copyOf from ArrayList.add -> presize / avoid copies
#   lock  : 60% blocked on one synchronized in a cache refresh        -> contention, not CPU

asprof -d 60 -e cpu --cstack no -f before.html <pid>
# ... deploy the fix, same load ...
asprof -d 60 -e cpu -f after.html <pid>        # compare, or use diff tools such as difffolded
```


© TechTribers. For personal use only. Do not redistribute.

Reading a flame graph: width = share of samples, height = call depth

1. Find the widest boxes near the TOP edge (self time).
2. Walk down: who calls it, and is that much work expected? 3. Compare before vs after (differential).



cpu = on-CPU work · wall = includes waiting (explains latency) · alloc = garbage · lock = contention

⚠️ **Interview trap**

Optimising from a **CPU** flame graph when requests are slow because threads are **waiting**. CPU profiles are blind to blocked time; take a wall-clock profile (or thread dumps) before concluding anything.

Q1155 PF-007 • Intermediate

Which jcmd, jstat, jmap and jinfo commands should you know for inspecting a running JVM?

⚡ **Quick answer**

`jcmd` is the Swiss-army tool (it talks to the JVM's diagnostic command interface), and replaces most single-purpose tools. Know: `VM.flags` / `VM.command_line` (effective configuration), `GC.heap_info` and `GC.class_histogram` (what is on the heap), `Thread.print`, `GC.heap_dump`, `VM.native_memory`, `JFR.start/dump`, and `jstat -gcutil` for a live GC view.

Terminal

```
jcmd -l                                   # JVMs on this host (pid and main class)
jcmd <pid> VM.command_line                # exact JVM options in use
jcmd <pid> VM.flags -all | grep -E "MaxHeapSize|UseG1GC|ParallelGCThreads"
jcmd <pid> VM.system_properties           # -D properties incl. file.encoding, user.timezone
jcmd <pid> VM.uptime
jcmd <pid> GC.heap_info                   # regions/generations and usage now
jcmd <pid> GC.class_histogram | head -20  # top classes by instances/bytes (forces no GC)
jcmd <pid> GC.heap_dump /tmp/heap.hprof   # full dump (pauses the app; mind disk space)
jcmd <pid> Thread.print -l > threads.txt  # thread dump with lock info
jcmd <pid> VM.native_memory summary       # needs -XX:NativeMemoryTracking
jcmd <pid> Compiler.codecache             # code cache usage
jstat -gcutil <pid> 1000 10               # S0 S1 E O M CCS YGC YGCT FGC FGCT GCT per second
jinfo -flag MaxHeapSize <pid>             # one flag value
jmap -histo:live <pid> | head             # histogram of LIVE objects (triggers a full GC!)
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> `jmap -histo:live` and `jcmd GC.class_histogram` differ: the first forces a **full GC** to count only live objects, the second (by default) doesn't. On a latency-sensitive production service, prefer the second.

📡 Q1156 PF-008 • Advanced

How do you profile continuously in production with an overhead budget?

> ⚡ **Quick answer**
> Keep a **low-overhead always-on recorder** so the data from an incident already exists: JFR in continuous mode with a ring buffer (`maxage` / `maxsize`) and the default settings (about 1% overhead), or a continuous-profiling backend (Pyroscope, Parca, Grafana Cloud Profiles, async-profiler agents). Dump the recording **on demand or on a trigger** (an alert, an OOM). Budget overhead explicitly (for example under 2% CPU) and verify it in a load test before enabling fleet-wide.

Treat profiles like logs: **retain them with the build version and pod name** so you can compare releases. The `profile` settings gather more detail at higher cost; use `default` continuously and `profile` for short, targeted recordings.

Terminal

```
# Always-on JFR: ring buffer of the last 30 minutes, dump when needed
JAVA_TOOL_OPTIONS="-XX:StartFlightRecording=name=cont,maxage=30m,maxsize=500m,\
disk=true,dumponexit=true,filename=/dumps/exit.jfr,settings=default \
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps"

jcmd <pid> JFR.dump name=cont filename=/dumps/incident-$(date +%s).jfr    # at alert time
jcmd <pid> JFR.configure                                                  # show buffers/settings

# Look at it later: JDK Mission Control (JMC), IntelliJ profiler, or `jfr print`
jfr print --events jdk.GarbageCollection,jdk.CPULoad /dumps/incident-*.jfr | head -50
```

🏋 Q1157 PF-009 • Advanced

How do you load-test a JVM service correctly (warm-up, steady state, coordinated omission)?

> ⚡ **Quick answer**
> A valid load test uses **production-like data and dependencies**, a **warm-up phase** that is discarded, a long enough **steady state**, and ramps to **saturation** to find the knee of the curve. The key trap is **coordinated omission**: a closed-loop client that waits for each response before sending the next **stops sending while the system stalls**, so it records few slow requests and under-reports tail latency. Use an **open workload model** with a fixed arrival rate (k6 `constant-arrival-rate` , Gatling open injection, wrk2).

© TechTribers. For personal use only. Do not redistribute.

Terminal

```
// k6: fixed arrival rate (open model): requests keep coming even if the server stalls
export const options = {
  scenarios: {
    steady: {
      executor: 'constant-arrival-rate',
      rate: 500, timeUnit: '1s',           // 500 requests per second, regardless of latency
      duration: '10m', preAllocatedVUs: 200, maxVUs: 1000,
    },
  },
  thresholds: { http_req_duration: ['p(95)<250', 'p(99)<600'], http_req_failed: ['rate<0.01'] },
};

# Procedure
#  1. warm up 3-5 minutes and discard it   2. steady state 10+ minutes at the target rate
#  3. step the rate up until p99 or errors break the SLO (find the saturation point)
#  4. soak test for hours: memory growth, connection leaks, GC drift
#  5. record JVM metrics alongside: GC pauses, heap after GC, thread counts, pool wait time
```

💡 **Pro tip**

Run the load generator on **separate machines** with enough CPU, and watch its own saturation. A load generator at 100% CPU produces beautiful, wrong latency numbers.

Q1158 PF-010 • Advanced

How do you find lock contention and thread starvation in a Java service?

⚡ **Quick answer**

Symptoms: low CPU, high latency, many threads in **BLOCKED** or **WAITING** state. Take **several thread dumps** a few seconds apart and look for many threads blocked on the **same monitor** (`waiting to lock <0x...>`) held by one thread; use a **JFR lock profile** or async-profiler `-e lock` to see contention time by call site. Fixes: shrink the critical section, **stripe or shard** locks (`ConcurrentHashMap` , `LongAdder`), use immutable or copy-on-write data, move slow work (I/O) **out** of locks, or replace a shared resource with a pool.

Terminal

```
jcmd <pid> Thread.print -l > t1.txt; sleep 5; jcmd <pid> Thread.print -l > t2.txt
grep -c "java.lang.Thread.State: BLOCKED" t1.txt                    # how many are blocked?
grep -A3 "waiting to lock" t1.txt | grep "<0x" | sort | uniq -c | sort -rn | head
#   38  - waiting to lock <0x00000006c2a8f1b0> (a com.acme.PriceCache)     <- one hot monitor
grep -B2 -A12 "locked <0x00000006c2a8f1b0>" t1.txt | head -30          # who holds it, doing what?
#   "pool-3-thread-7" RUNNABLE ... at java.net.SocketInputStream.socketRead0   <- I/O inside the
#   lock

asprof -d 30 -e lock -f lock.html <pid>                                # contended time by call site
jfr print --events jdk.JavaMonitorEnter /tmp/app.jfr | head -30          # JFR lock events
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

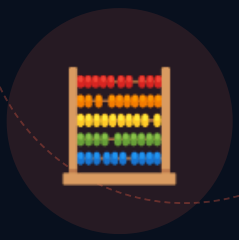
Raising pool sizes to "fix" blocked threads. More threads just means more threads waiting on the same lock. Fix the **contended resource**, not the symptom.

Section 02

Heap and Memory Configuration

How to size the heap from the live set, whether -Xms should equal -Xmx, the full memory footprint of a JVM process, Native Memory Tracking, metaspace, thread stacks, direct memory, compressed oops and the 32 GB boundary, huge pages and practical footprint reduction.

Questions 1159–1168 · 10 questions



Q1159 PF-011 • Intermediate

How do you size the Java heap (-Xms, -Xmx) for a service?

⚡ **Quick answer**

Size from **measured data**, not folklore. Find the **live set**: the heap occupied **after** a full or concurrent GC cycle at peak load. Give the heap **roughly 2–4× the live set**, so GC has room to work and young collections stay infrequent; then check **GC overhead** (target under 5–10% of time) and **allocation rate** against the pause SLO and adjust. Too small: frequent GCs, promotion failures, OOM. Too large: longer full collections, wasted memory, and in containers an OOM-kill risk.

The old-generation occupancy after GC is a **trend to alert on**: a steadily rising floor across GCs is a leak, a stable floor at 60–70% of the old gen means the heap is too small for the workload. Q1161 covers the rest of the process memory, which `-Xmx` does not limit.

Terminal

```
# 1. Measure the live set under realistic load (heap AFTER GC is what matters)
jstat -gcutil <pid> 5000 | awk '{print $4, $5}'           # O (old gen %), M (metaspace %) trend
jcmd <pid> GC.heap_info
# GC log line shows the live data directly:
#   [gc] GC(412) Pause Young (Normal) 1536M->412M(2048M) 11.3ms
#                                     before->AFTER(committed)  => live set ~ 412 MB

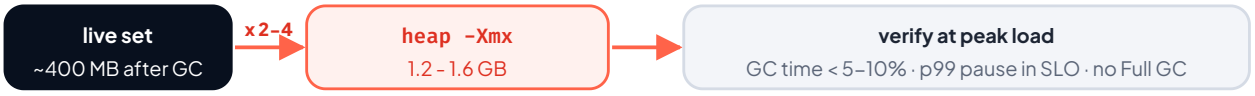
# 2. Rule of thumb: Xmx = 2-4 x live set (leave headroom for spikes and fragmentation)
#    live set 400 MB  ->  -Xmx1200m to 1600m

# 3. Verify: GC overhead and pause time at peak load
#    GC time share < 5%, p99 pause within SLO, no "Pause Full"/to-space exhausted
```


© TechTribers. For personal use only. Do not redistribute.

```
# 4. Re-check after each release: live set grows with caches, sessions and data volume
java -Xms1500m -Xmx1500m -XX:+UseG1GC -Xlog:gc*:file=gc.log:time,uptime -jar app.jar
```

Sizing the heap from the live set (measured AFTER GC at peak load)



Too small: frequent GC, premature promotion, Full GC, OOM.
Too large: longer old-gen work, wasted memory, OOMKill risk in containers.
Rising old-gen floor across GCs = leak. Flat floor at 60–70% of old gen = heap too small.
Re-check each release: the live set grows with caches, sessions and data.

Q1160 PF-012 • Intermediate

Should -Xms equal -Xmx, and what does -XX:+AlwaysPreTouch do?

⚡ **Quick answer**

Setting `-Xms` = `-Xmx` gives a fixed-size heap: no resize pauses, predictable footprint, and one less thing for the GC to decide. The cost is committing memory at startup even if the app doesn't need it (and with ZGC/G1 it disables heap shrinking). `-XX:+AlwaysPreTouch` touches every heap page at startup so the OS commits physical memory then rather than on first use, removing **page-fault latency spikes** in production at the price of **slower startup** (seconds for big heaps).

Terminal

```
# Latency-sensitive service on dedicated memory: fixed heap, pre-touched
java -Xms8g -Xmx8g -XX:+AlwaysPreTouch -XX:+UseZGC -jar trading.jar

# Many small services in Kubernetes: let the container limit decide, keep startup fast
java -XX:InitialRAMPercentage=50 -XX:MaxRAMPercentage=75 -jar orders.jar

# Check what was applied
java -XX:+PrintFlagsFinal -version | grep -E "InitialHeapSize|MaxHeapSize|AlwaysPreTouch"
```

⚠️ **Interview trap**

`AlwaysPreTouch` with a large heap in a pod with a short readiness timeout: startup takes longer than the probe allows, and Kubernetes restarts the pod in a loop. Account for it in probe settings (Q1193).

© TechTribers. For personal use only. Do not redistribute.

Q1161 PF-013 • Advanced

Why does a JVM process use more memory than -Xmx? (the full footprint)

⚡ **Quick answer**

`-Xmx` limits only the **Java heap**. The process also holds **Metaspace** (class metadata), the **code cache** (JIT-compiled code), **thread stacks** (1 MB reserved each by default), **direct/off-heap buffers** (NIO, Netty), **GC data structures** (remembered sets, mark bitmaps: several % of the heap, more for G1), the **JIT compiler's working memory**, **symbol and string tables**, and native libraries. Typical total: heap plus **25–50%**, more with many threads or large direct buffers.

This is the single most common cause of `OOMKilled` **pods with a healthy heap** (Q338): the heap was fine, the process exceeded the cgroup limit because of everything else. Leave 25–40% of the container limit outside `-Xmx` (`MaxRAMPercentage` 60–75 is a common range) and verify with NMT and RSS.

Terminal

```
# Rough budget for a 2 GB heap service (numbers are typical, measure yours with NMT)
#   Java heap (-Xmx2g)                          2048 MB
#   Metaspace (8,000 classes)                     90 MB
#   Code cache (JIT output)                       60 MB
#   Thread stacks (200 threads x 1 MB reserved)   up to 200 MB (committed usually less)
#   GC structures (G1 remembered sets, bitmaps)  ~150 MB
#   Direct buffers (Netty / NIO)                  64 MB
#   Compiler, symbol tables, libc arenas, misc    ~100 MB
#   ------------------------------------------------------------
#   Process RSS                                  ~2.7 GB   -> a 2.5 GB container limit gets
#   OOMKilled

# Compare what the OS sees with what the JVM reports
ps -o pid,rss,vsz,cmd -p <pid>              # RSS: resident set the kernel counts against cgroup
jcmd <pid> VM.native_memory summary         # JVM's own breakdown (needs NMT, next question)
```

A 2 GB heap is not a 2 GB process (typical, measure yours with NMT)



Java heap -Xmx2g	~2048 MB
GC structures	~150 MB
Thread stacks	~200 MB
Metaspace	~90 MB
Code cache	~60 MB
Direct buffers	~64 MB
Misc native	~100 MB

Process RSS ≈ 2.7 GB

→ a 2.5 GB container limit gets OOMKilled

Set MaxRAMPercentage 60–75 and leave the rest for everything outside the heap.

© TechTribers. For personal use only. Do not redistribute.

Q1162 PF-014

• Advanced

How do you use Native Memory Tracking (NMT) to find where non-heap memory goes?

⚡ **Quick answer**

Start the JVM with `-XX:NativeMemoryTracking=summary` (or `detail`), then query it with `jcmd <pid> VM.native_memory`. It breaks JVM-internal native memory into categories (Java Heap, Class, Thread, Code, GC, Compiler, Internal, Symbol, Arena Chunk, ...), and `baseline` + `summary.diff` shows **growth over time**, which is how you find a native leak. It adds a little overhead (about 5–10% with `summary`) and covers memory the JVM allocates, not memory allocated by native libraries.

Terminal

```
java -XX:NativeMemoryTracking=summary -jar app.jar

jcmd <pid> VM.native_memory baseline                # remember the current state
# ... run load for an hour ...
jcmd <pid> VM.native_memory summary.diff            # what grew since the baseline?
#  -                 Thread (reserved=412MB +96MB, committed=95MB +44MB)    <- thread leak
#                      (threads: 410 +120)
#  -                 Class (reserved=1112MB, committed=96MB)
#  -              Internal (reserved=180MB +140MB, committed=180MB +140MB)   <- often direct buffers

jcmd <pid> VM.native_memory detail | grep -A3 "Thread"    # call sites of allocations (detail mode)

# Memory not tracked by NMT: JNI libraries, glibc malloc arenas, mmap'd files
#   -> compare NMT total with RSS; set MALLOC_ARENA_MAX=2 to limit glibc arena fragmentation
```

💡 **Pro tip**

A big gap between RSS and the NMT total usually means **glibc malloc arena fragmentation** or a native library. `MALLOC_ARENA_MAX=2` (or switching the allocator to jemalloc) is a well-known fix for containers whose RSS keeps creeping up.

Q1163 PF-015

• Intermediate

How do you size and protect Metaspace (MaxMetaspaceSize, class-loader leaks)?

⚡ **Quick answer**

Metaspace stores class metadata in native memory and, by default, is **unbounded**, so a class-loader leak (repeated redeploys, dynamic proxies or generated classes created without limit) grows until the machine or container runs out. Set `-XX:MaxMetaspaceSize` (typically 256–512 MB for a Spring Boot service; measure `jcmd VM.metaspace`) so a leak becomes a clear `OutOfMemoryError: Metaspace` instead of an OOM-kill. `MetaspaceSize` only sets the threshold of the first GC-triggering resize.

© TechTribers. For personal use only. Do not redistribute.

```
Terminal

# Protect the process: a leak fails fast with a clear error
java -XX:MaxMetaspaceSize=384m -XX:+ExitOnOutOfMemoryError -jar app.jar

jcmd <pid> VM.metaspace                       # used/committed by class loader
jcmd <pid> GC.class_stats                     # (diagnostic) class count and sizes
jstat -gcutil <pid> 5000                    # M and CCS columns: metaspace and compressed class space
java -Xlog:class+load:file=classes.log -jar app.jar     # which classes load, and from where

# Typical leak sources
#   - hot redeploy in an app server (old ClassLoader kept alive by a ThreadLocal or static)
#   - generating classes per request: CGLIB/ByteBuddy proxies, Groovy scripts, reflection accessors
#   - frameworks that define a new class loader per tenant or plugin
```

⚠️ **Interview trap**

Setting `MaxMetaspaceSize` far below the real need (a few dozen MB): the app dies at startup or under load with `OutOfMemoryError: Metaspace`. Measure the steady value first, then add 50–100% headroom.

🧵 **Q1164** PF-016 • Intermediate

How does thread count affect memory, and when do you tune -Xss?

⚡ **Quick answer**

Each platform thread reserves a stack (**1 MB** by default on 64-bit Linux, set by `-Xss` or `ThreadStackSize`) in native memory. Reserved memory becomes **resident only as the stack is used**, but 500–2,000 threads still cost hundreds of MB of address space and measurable RSS. Reduce the number of threads (right-size pools) before shrinking stacks; lower `-Xss` only if you've measured it safe, because **deep recursion or large frames then throw** `StackOverflowError`. **Virtual threads** keep their stacks on the **heap** as small, growable objects.

With virtual threads (Q379) the per-request thread cost disappears, but you still bound **concurrency to scarce resources** (database connections, downstream rate limits) with semaphores or pools, not with thread counts.

```
Terminal

java -Xss512k -jar app.jar                  # smaller stacks (test deep call chains first!)
java -XX:ThreadStackSize=512 -jar app.jar         # same, in KB

jcmd <pid> Thread.print | grep -c '^"'            # thread count
cat /proc/<pid>/status | grep -E "Threads|VmRSS|VmHWM"

# Where do 800 threads come from in a Spring Boot service?
#   Tomcat worker pool        200 (server.tomcat.threads.max)
#   HikariCP housekeeping      2-3
#   Scheduler / async pools    8-50 each, often unbounded by default (@Async
#   SimpleAsyncTaskExecutor!)
```


© TechTribers. For personal use only. Do not redistribute.

```
#   Kafka clients, gRPC, Netty event loops: 2 x cores each
#   GC and JIT compiler threads: scale with CPU count (see the container CPU question)
```

📦 **Q1165** PF-017 • Intermediate

How do you size and monitor direct (off-heap) memory, and what happens when it runs out?

⚡ **Quick answer**

Direct `ByteBuffer`s (NIO, Netty, Kafka clients, some JDBC drivers) allocate **native memory outside the heap**, capped by `-XX:MaxDirectMemorySize` (defaults to roughly the max heap size). Exhausting it throws `OutOfMemoryError: Direct buffer memory`, and because freeing depends on **GC cleaning up the small heap objects** that own the buffers, a quiet heap can let direct memory pile up. Pooled allocators (Netty) hold memory deliberately; leaks show up as steadily rising direct usage.

Terminal

```
java -XX:MaxDirectMemorySize=256m -jar app.jar   # fail with a clear error, not an OOM-kill

# Observe it
jcmd <pid> VM.native_memory summary | grep -i -A2 "Internal\|Other"
# JMX / Micrometer metric:  jvm.buffer.memory.used{id="direct"}   jvm.buffer.count{id="direct"}

# Netty: see allocator usage and enable leak detection while testing
java -Dio.netty.leakDetection.level=PARANOID -Dio.netty.maxDirectMemory=0 -jar app.jar
#   maxDirectMemory=0: Netty uses its own accounting and reports pooled arena sizes

# Typical causes: unreleased ByteBuf (missing release()), huge responses buffered in memory,
# many connections each holding pooled buffers
```

💡 **Pro tip**

Alert on `jvm.buffer.memory.used` for `direct` the same way you alert on heap. It is the early warning for off-heap leaks that otherwise end as an OOM-kill with a quiet heap.

© TechTribers. For personal use only. Do not redistribute.

Q1166 PF-018

• Advanced

What are compressed oops and compressed class pointers, and why does crossing 32 GB of heap cost memory?

⚡ Quick answer

With **compressed oops**, object references are stored as 32-bit values and scaled by the 8-byte object alignment, so a heap up to about **32 GB** needs only 4 bytes per reference instead of 8 (halving pointer overhead and improving cache use). Beyond roughly 32 GB the JVM switches to 64-bit references, and the **same live data takes about 10–20% more memory**: a 36 GB heap can hold less than a 31 GB one. **Compressed class pointers** do the same for class metadata pointers. Both are on by default; check with `PrintFlagsFinal`.

Practical advice: **stay at or below ~31 GB** per JVM, or jump well beyond (48 GB+) so the gain outweighs the 64-bit overhead. Several 16–24 GB JVMs often beat one 64 GB JVM (shorter GC, fault isolation), at the price of memory duplicated per process. **Compact object headers** (Q317) shrink every object further.

Terminal

```
java -XX:+PrintFlagsFinal -version | grep -E "UseCompressedOops|UseCompressedClassPointers"
#   bool UseCompressedOops          = true
#   bool UseCompressedClassPointers = true

java -Xmx31g  -XX:+PrintFlagsFinal -version | grep UseCompressedOops      # true
java -Xmx40g  -XX:+PrintFlagsFinal -version | grep UseCompressedOops      # false: 64-bit refs

# The exact boundary depends on alignment and platform: check the log
java -Xmx30g -Xlog:gc+init -version | grep -i "compressed"
#   Compressed Oops: Enabled (Zero based)

# Larger objects: -XX:ObjectAlignmentInBytes=16 extends compressed oops to ~64 GB (more
#   padding)
```

Q1167 PF-019

• Advanced

When do huge pages (Transparent Huge Pages, -XX:+UseLargePages) help a JVM, and what are the risks?

⚡ Quick answer

Memory is mapped in 4 KB pages; a large heap needs millions of page-table entries and many **TLB misses**. **Huge pages** (2 MB) cut that overhead, typically giving a **few percent** throughput gain on large heaps with random access. Options: `-XX:+UseTransparentHugePages` (the kernel promotes pages automatically; easy, but the kernel may stall to compact memory) or `-XX:+UseLargePages` with reserved **hugetlbfs** pages (predictable, needs OS setup). Combine with `AlwaysPreTouch` and fixed `Xms = Xmx`.

© TechTribers. For personal use only. Do not redistribute.

Terminal

```
# Transparent Huge Pages: no OS reservation needed, kernel setting must allow it
cat /sys/kernel/mm/transparent_hugepage/enabled        # always | madvise | never
java -XX:+UseTransparentHugePages -Xms16g -Xmx16g -XX:+AlwaysPreTouch -jar app.jar

# Explicit huge pages: reserve them first
sysctl -w vm.nr_hugepages=9000                         # 9000 x 2 MB = 18 GB
java -XX:+UseLargePages -Xms16g -Xmx16g -jar app.jar

# Verify they are used
grep -i huge /proc/meminfo; grep -i AnonHugePages /proc/<pid>/smaps_rollup
```

⚠️ **Interview trap**

Turning THP to `always` system-wide for every workload. Latency-sensitive services can see sporadic stalls when `khugepaged` compacts memory, and small containers may inflate RSS. Use `madvise` mode with the JVM flag, and benchmark before and after.

Q1168 PF-020 • Advanced

How do you reduce the memory footprint of a Java service?

⚡ **Quick answer**

Attack the biggest consumers in order. **Heap**: reduce the live set (find retained size with a heap dump, drop unneeded caches, store data compactly: primitive arrays, `byte[]`, enums and interned or shared strings), switch to **compact object headers** (`-XX:+UseCompactObjectHeaders` where available, Q317), and enable `-XX:+UseStringDeduplication` with G1 for string-heavy heaps. **Non-heap**: fewer threads, `MaxMetaspaceSize`, a smaller code cache, **class-data sharing** (AppCDS shares class metadata across JVMs on a host), and the right GC for tiny containers (Serial uses the least memory).

Footprint work trades **CPU or peak performance** for memory (Serial GC, C1-only, smaller heaps), so tie it to a goal such as "fit 3 pods per node instead of 2" and re-run the load test. Don't micro-optimise a 200 MB live set while a 1.5 GB container limit is the real driver.

Terminal

```
# Small service in a 512 MB container
java -XX:+UseSerialGC -Xss512k -XX:MaxRAMPercentage=65 \
     -XX:MaxMetaspaceSize=128m -XX:ReservedCodeCacheSize=64m \
     -XX:TieredStopAtLevel=1 -Xshare:auto -jar app.jar        # C1 only: less memory, lower peak

# Mid-size service: shrink objects and strings
java -XX:+UseCompactObjectHeaders -XX:+UseStringDeduplication -XX:+UseG1GC -jar app.jar

# Find what dominates the heap (histogram, then a dump for retained sizes)
jcmd <pid> GC.class_histogram | head -15
#   1:  5210384  250,098,432  [B                      <- byte[]: strings, payloads
#   2:  1630221   52,167,072  java.util.HashMap$Node  <- maps: consider primitive/compact
```


© TechTribers. For personal use only. Do not redistribute.

```
#   structures

# Code-level: Integer in collections ~20 bytes/element vs 4 for int[] (measured in Part 16)
```

Section 03

Garbage Collection Tuning in Practice

The G1 knobs that matter and the ones to leave alone, humongous objects, mapping GC symptoms to causes and fixes, allocation rate and premature promotion, Parallel GC for batch work, ZGC and Shenandoah settings, GC behaviour under container CPU limits and reducing garbage in code.

Questions 1169–1177 · 9 questions



Q1169 PF-021 • Advanced

Which G1 tuning flags matter (MaxGCPauseMillis, region size, IHOP), and which should you leave alone?

⚡ Quick answer

G1 is largely **self-tuning**: the main input is the **pause-time goal** `-XX:MaxGCPauseMillis` (default 200 ms), a target, not a guarantee. Raising heap size or lowering allocation usually helps more than flags. Occasionally useful: `-XX:G1HeapRegionSize` (to avoid humongous objects), `-XX:InitiatingHeapOccupancyPercent` when concurrent cycles start too late (adaptive by default), and `-XX:G1ReservePercent` for to-space headroom. **Avoid** fixing young-gen size (`-Xmn` , `NewRatio`): it disables G1's ability to meet the pause goal.

Terminal

```
# Start simple: heap + pause goal, then measure
java -Xms4g -Xmx4g -XX:+UseG1GC -XX:MaxGCPauseMillis=100 \
     -Xlog:gc*:file=gc.log:time,uptime -jar app.jar

# Large objects (e.g. 4-10 MB payloads) cause humongous allocations: enlarge regions
java -XX:G1HeapRegionSize=16m ...                    # 1-32 MB, power of two

# Concurrent cycle starts too late -> Full GC / to-space exhausted: start earlier, add reserve
java -XX:InitiatingHeapOccupancyPercent=35 -XX:G1ReservePercent=15 ...

# Give concurrent marking more threads if marking can't keep up with allocation
java -XX:ConcGCThreads=4 ...
```


© TechTribers. For personal use only. Do not redistribute.

```
# Leave alone unless you have measured a reason:
#   -Xmn / -XX:NewSize / -XX:NewRatio (fixes young size; breaks pause-goal adaptation)
#   -XX:SurvivorRatio, -XX:MaxTenuringThreshold, -XX:G1MixedGCCountTarget, ParallelGCThreads
```

⚠️ **Interview trap**

Copying a 40-line flag set from a blog post written for CMS or Java 8. Most of those flags are obsolete or harmful on G1 today. Start from defaults plus heap size, change **one** flag, and compare GC logs.

Q1170 PF-022 • Advanced

What are humongous objects in G1, why do they hurt, and how do you fix them?

⚡ **Quick answer**

In G1 an object of **half a region or more** is **humongous**: it's allocated directly into contiguous **old-generation regions**, bypassing the young generation, and is reclaimed only at the end of a concurrent cycle (or early by *eager reclaim* if it has no references). Many humongous allocations cause **fragmentation, premature concurrent cycles and Full GCs**. Fixes: **increase** `G1HeapRegionSize` so objects fall below the limit, **stream or chunk** large payloads, reuse buffers, and avoid building giant arrays or strings.

Typical culprits are **large** `byte[]` **payloads** (file uploads, report PDFs), giant `StringBuilder`s, and `ArrayList`s that grow by repeated copying to tens of megabytes. Region size is chosen from the heap size at startup, so a service that is moved to a **smaller container** can suddenly start producing humongous allocations.

Terminal

```
# GC log: humongous allocations trigger concurrent cycles
#   [gc] GC(87) Pause Young (Concurrent Start) (G1 Humongous Allocation) 3020M->1822M(4096M) 14.7ms
#                                              ^^^^^^^^^^^^^^^^^^^^^^^^ cause

# Region size is heap/2048 rounded to a power of two (1-32 MB): a 4 GB heap uses 2 MB regions,
# so any object >= 1 MB is humongous.
java -Xlog:gc+humongous=debug -jar app.jar          # log each humongous allocation
jcmd <pid> GC.heap_info                             # region size and counts

# Fixes
java -XX:G1HeapRegionSize=16m -jar app.jar          # 8 MB objects are no longer humongous

# Code: avoid huge single allocations
#   - read files and HTTP bodies as streams, not readAllBytes() of 200 MB
#   - presize collections sensibly instead of growing a giant array by copying
#   - use pooled, fixed-size buffers for I/O
```


© TechTribers. For personal use only. Do not redistribute.

Q1171 PF-023 • Advanced

Diagnose these GC symptoms: frequent young GCs, long pauses, Full GC, to-space exhausted, allocation stalls.

⚡ Quick answer

Map each symptom to its likely cause. **Very frequent young GCs**: high **allocation rate** or a small young generation. **Long young pauses**: large survivor/promotion volume, too many GC threads fighting for few CPUs, or swapping. **Full GC**: old gen too full (heap too small, a leak, humongous objects, concurrent cycle started too late). **To-space exhausted / evacuation failure** (G1): no free regions to copy into: more headroom (`G1ReservePercent` , bigger heap), earlier concurrent cycles. **Allocation stalls** (ZGC): the heap is too small for the allocation rate. **Rising old-gen floor**: a leak.

Always read symptoms **together with the heap-after-GC trend** (live set) and the **allocation rate** (MB/s). The same Full GC means "heap too small" for a flat live set and "leak" for a rising one.

Terminal

```
# What each log line tells you
#  Pause Young (Normal)  (G1 Evacuation Pause) 1536M->412M(2048M) 11ms       healthy
#  Pause Young (Concurrent Start) (G1 Humongous Allocation)                   humongous objects
#  Pause Young (Prepare Mixed)                                                old regions to clean
#  To-space exhausted                                                         evacuation failed:
#   raise heap/reserve
#  Pause Full (G1 Compaction Pause) 3900M->3100M(4096M) 2.4s                  heap far too small
#   / leak
#  Allocation Stall (ZGC)  thread ... stalled 38 ms                           ZGC cannot keep up:
#   more heap
#  Concurrent Mark Cycle  frequent and long                                   live set near the
#   heap size

# Quick quantification from a log (GCViewer, GCeasy, or JFR)
grep -c "Pause Young" gc.log;  grep -c "Pause Full" gc.log
grep "Pause" gc.log | sed -E 's/.* ([0-9.]+)ms$/\1/' | sort -n | tail -5       # worst pauses

# Fix order: (1) more heap or less allocation  (2) check for a leak  (3) humongous  (4) flags
```

GC symptom → likely cause → first fix

Very frequent young GCs	high allocation rate	cut garbage; check allocation profile
Long young pauses	big promotion volume, too few CPUs	more heap/CPU; reduce survivors
Pause Full / Full GC	heap too small, leak, humongous	check old-gen floor; region size
To-space exhausted (G1)	no free regions to copy into	more heap, G1ReservePercent
Allocation Stall (ZGC)	allocating faster than reclaim	add heap; keep 20–30% free
Old-gen floor rising	memory leak	histogram diff, heap dump

Read symptoms together with heap-after-GC (live set) and allocation rate (MB/s).

© TechTribers. For personal use only. Do not redistribute.

Q1172 PF-024

• Advanced

How do allocation rate and premature promotion affect GC, and how do you reduce them?

⚡ Quick answer

The **allocation rate** (MB/s of new objects) sets how often young GCs run. Short-lived objects die in the young generation at almost no cost; but if the young generation is too small or the rate too high, live objects get **promoted early** (premature promotion) into the old generation, which then fills with garbage and triggers expensive old-gen work. Reduce garbage at the source (fewer temporary objects per request), and watch the **promotion rate** in logs. Measure allocation with JFR (`jdk.ObjectAllocationSample`) or an allocation flame graph.

Terminal

```
# Allocation rate from the GC log: (young size after previous GC -> before next) / time
#  GC(412) 1536M->412M   at 10.2 s
#  GC(413) 1536M->415M   at 10.9 s     => ~1,100 MB allocated in 0.7 s = ~1.5 GB/s: very high

jfr print --events jdk.GCHeapSummary,jdk.ObjectAllocationSample app.jfr | head -40
asprof -d 60 -e alloc -f alloc.html <pid>              # who allocates

# Typical garbage generators (fix in this order)
#   1. boxing in collections/streams: Map<Long,Long>, Stream<Integer>, autoboxing in loops
#   2. String concatenation, String.format, regex matching and substring in hot paths
#   3. per-request copies: JSON -> Map -> DTO -> JSON; ModelMapper-style conversions
#   4. Arrays.copyOf growth: ArrayList/StringBuilder without initial capacity
#   5. exceptions used for control flow (stack traces), logging with eager toString()
```

💡 Pro tip

Target the **top 3 allocation sites** in the flame graph; the allocation profile is heavily skewed, and fixing three hot spots often halves the rate. Don't chase 1% contributors.

Q1173 PF-025

• Intermediate

When is Parallel GC a better choice than G1 (batch and throughput workloads)?

⚡ Quick answer

Parallel GC (`-XX:+UseParallelGC`) uses multiple threads for **stop-the-world** young and old collections and has the **lowest GC overhead and no concurrent-collector barriers or memory cost**, so it often gives the highest **throughput** for **batch jobs, ETL and offline computation** where long pauses don't matter. G1 trades some throughput for bounded pauses, which is the right trade for request/response services. Parallel's pauses grow with heap size, so it suits moderate heaps or jobs that tolerate multi-second pauses.

© TechTribers. For personal use only. Do not redistribute.

Decision rule: **services with latency SLOs → G1 (or ZGC for strict pauses); batch and throughput-bound jobs → Parallel; tiny containers → Serial**. Re-test after each JDK upgrade: collectors improve, and G1's throughput gap to Parallel has been shrinking.

Terminal

```
# Nightly batch: maximise throughput, ignore pause times
java -XX:+UseParallelGC -Xms16g -Xmx16g -XX:ParallelGCThreads=8 -jar batch-job.jar

# Compare collectors on the SAME workload and measure the job's total runtime
for gc in UseParallelGC UseG1GC UseZGC; do
  /usr/bin/time -v java -XX:+$gc -Xmx16g -jar batch-job.jar 2> run-$gc.txt
done
grep -E "Elapsed|Maximum resident" run-*.txt

# Tuning Parallel GC: throughput goal and GC time share
java -XX:+UseParallelGC -XX:GCTimeRatio=19 ...         # aim for <= 5% of time in GC (1/(1+19))
# In containers with <2 CPUs or <1792 MB the JVM picks Serial GC by default
```

Q1174 PF-026 • Advanced

How do you tune ZGC (SoftMaxHeapSize, heap headroom, allocation stalls, uncommit)?

⚡ **Quick answer**

ZGC does almost all work **concurrently**, so pauses stay sub-millisecond to low-millisecond regardless of heap size, but it needs **headroom**: if the application allocates faster than ZGC can reclaim, threads hit **allocation stalls**. Tune by **adding heap** (the most effective lever), then **concurrent GC threads** (`-XX:ConcGCThreads`), and use `-XX:SoftMaxHeapSize` to keep the working heap smaller than `Xmx` (ZGC exceeds it only to avoid stalls). ZGC can **uncommit** unused memory (`ZUncommitDelay`). ZGC is generational by default in current JDKs (the non-generational mode was removed in JDK 24).

Terminal

```
# Low-latency service: generous heap, fixed size
java -XX:+UseZGC -Xms12g -Xmx12g -XX:SoftMaxHeapSize=9g -jar app.jar
#   SoftMaxHeapSize: ZGC tries to stay below 9 GB and grows toward 12 GB only under pressure

# JDK 21-22: generational mode was opt-in
java -XX:+UseZGC -XX:+ZGenerational -jar app.jar          # JDK 23+: generational is the default

# Observe: allocation stalls and heap headroom
java -Xlog:gc*,gc+stall=info:file=zgc.log:time,uptime -jar app.jar
#   Allocation Stall (http-nio-8080-exec-14) 38.211ms       <- need more heap or fewer allocations

jcmd <pid> GC.heap_info
# Memory returned to the OS when idle (disabled automatically if Xms == Xmx)
java -XX:+ZUncommit -XX:ZUncommitDelay=300 ...
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Sizing ZGC like G1, with the heap just above the live set. ZGC's concurrent cycles need free memory to run in; with too little, throughput collapses into stalls. Give it **at least 20–30% free heap** at steady state and watch for `Allocation Stall`.

⚖️ **Q1175** PF-027 • Advanced

ZGC vs Shenandoah: how do you choose between the low-pause collectors?

⚡ **Quick answer**

Both are **concurrent, mostly pause-free** collectors. **ZGC** (Oracle-led, colored pointers and load barriers) scales to very large heaps (terabytes) with consistently tiny pauses and is available in all major JDK builds. **Shenandoah** (Red Hat-led, Brooks-style forwarding, with **generational mode** as a product feature from JDK 25) is popular in **Red Hat, Temurin and Corretto** builds, works well on small to medium heaps, and has its own tuning modes. Choose on **measured behaviour for your workload** and what your JDK vendor supports; both beat G1 on tail latency, at some throughput and memory cost.

The honest answer in an interview: start with **G1**; move to **ZGC** if you have a latency SLO that G1 can't meet and enough memory headroom; consider **Shenandoah** if your vendor's JDK favours it. Whatever you pick, **prove it with your own load test**, not with a benchmark from another workload.

Terminal

```
# ZGC
java -XX:+UseZGC -Xmx16g -jar app.jar

# Shenandoah (check that your JDK build includes it: java -XX:+PrintFlagsFinal -version | grep
#   Shenandoah)
java -XX:+UseShenandoahGC -Xmx16g -jar app.jar
java -XX:+UseShenandoahGC -XX:ShenandoahGCMode=generational -Xmx16g -jar app.jar   # JDK 25+

# Compare under identical load, collecting GC logs and application percentiles
#   metrics: p99 / p99.9 latency, throughput at SLO, CPU utilisation, RSS (barriers cost memory/CPU)
java -Xlog:gc*:file=gc-$COLLECTOR.log -XX:+Use${COLLECTOR}GC ...
```


© TechTribers. For personal use only. Do not redistribute.

Q1176 PF-028

• Advanced

How do container CPU limits affect GC thread counts and the choice of collector?

⚡ Quick answer

The JVM reads the container's **CPU quota** and derives `availableProcessors()` : a limit of 1.5 CPUs becomes 2, less than 1 becomes 1. GC and compiler thread counts, **fork-join common pool** size and many library defaults scale from that number. With **fewer than 2 CPUs or under ~1.8 GB** the JVM treats the machine as *not server-class* and defaults to the **Serial** collector. A hard CPU limit also causes **CFS throttling**: GC and JIT threads burn the quota in bursts and all application threads are paused until the next period.

Give latency-sensitive JVMs a **CPU limit at least as large as the burst they need** (or no limit with a request, if your platform allows), and avoid fractional limits below 2 CPUs for services with real GC activity. Details for Kubernetes are in Q1180.

Terminal

```
# What does the JVM think it has?
jcmd <pid> VM.flags | tr ' ' '\n' | grep -E "ParallelGCThreads|ConcGCThreads|CICompilerCount"
java -XX:+PrintFlagsFinal -version | grep -E "ActiveProcessorCount|UseContainerSupport"
java -Xlog:os+container=info -version                  # shows detected cgroup CPU/memory limits

# Pin the number explicitly when limits and requests differ, or for predictable pool sizing
java -XX:ActiveProcessorCount=4 -jar app.jar

# Make the collector choice explicit; do not rely on ergonomics in containers
java -XX:+UseG1GC -jar app.jar                         # even on a 1-CPU pod, if that is a decision

# Throttling check inside the pod: nr_throttled and throttled_time rising = quota exhausted
cat /sys/fs/cgroup/cpu.stat                            # cgroup v2: nr_throttled, throttled_usec
```

Q1177 PF-029

• Advanced

How do you reduce GC pressure in code? (pooling myths, hot-path allocations, escape analysis)

⚡ Quick answer

Allocation in Java is cheap (a pointer bump in a TLAB, Q322), and **short-lived objects cost almost nothing to collect**, so the goal isn't "no allocation" but **no avoidable allocation on hot paths**. Highest return: remove **boxing** (`Map<Long,...>` , `Stream<Integer>`), **reuse immutable or expensive-to-build objects** (`Pattern` , `DateTimeFormatter` , `ObjectMapper`), **presize** collections and builders, avoid per-request conversions, and parse streams instead of materialising whole documents. **Object pools** usually make things worse (retention, contention, bugs) except for genuinely expensive or native resources (buffers, connections).

© TechTribers. For personal use only. Do not redistribute.

```java
// 1. Boxing in a hot loop: ~16 bytes of garbage per iteration
long sum = 0;
for (Long v : values) sum += v;                // values is List<Long>: unboxing, plus boxed storage
long[] primitive = values.stream().mapToLong(Long::longValue).toArray();   // iterate this instead

// 2. Rebuilt per call: shared, immutable and thread-safe
private static final Pattern ORDER_ID = Pattern.compile("^ORD-(\\d{8})$");
private static final DateTimeFormatter DAY = DateTimeFormatter.ofPattern("yyyy-MM-dd");
private static final ObjectMapper MAPPER = new ObjectMapper();    // thread-safe once configured

// 3. Presize: avoids repeated Arrays.copyOf during growth
List<OrderDto> out = new ArrayList<>(orders.size());
// Java 19+: sizes for the load factor
Map<String, Integer> index = HashMap.newHashMap(expectedSize);
StringBuilder sb = new StringBuilder(256);

// 4. Don't pool small objects: the pool retains them across GCs (promotion) and adds contention
// Pooling IS justified for direct buffers, connections, and very large reusable arrays
```

💡 **Pro tip**

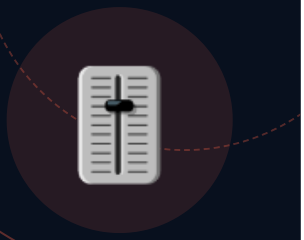
Confirm with an **allocation profile before and after**. If the fix doesn't move the allocation rate or the p99, revert it: simpler code that the JIT can already optimise (escape analysis, Q313) beats clever code.

© TechTribers. For personal use only. Do not redistribute.

Section 04

JVM Flags, System Properties and Runtime Configuration

Where JVM options come from and which wins, a production baseline for Spring Boot on Kubernetes, requests and limits for JVMs, the system properties and security settings that matter (encoding, time zone, TLS, DNS, entropy), flags that no longer exist, module flags, and launching the JVM correctly in Docker.



Questions 1178–1188 · 11 questions

Q1178 PF-030

• Intermediate

What kinds of JVM options exist, and where can you set them (command line, JAVA_TOOL_OPTIONS, JDK_JAVA_OPTIONS, argument files)?

⚡ Quick answer

Option families: **standard** (`-version` , `-cp` , `-D`), `-X` (non-standard: `-Xmx` , `-Xss` , `-Xlog`), and `-XX` HotSpot flags, which are `product` (supported), `diagnostic` (need `-XX:+UnlockDiagnosticVMOptions`), `experimental` (need `-XX:+UnlockExperimentalVMOptions`) or `manageable` (changeable at run time). Sources: the **command line**, `JAVA_TOOL_OPTIONS` (read by every JVM; prints "Picked up ..."), `JDK_JAVA_OPTIONS` (only the `java` launcher, JDK 9+) and `@argfiles` . Later sources can override earlier ones, so the **command line wins**. `JAVA_OPTS` is **not** read by the JVM at all.

Terminal

```
java -XX:+PrintFlagsFinal -version | head -5        # every flag, its value, and its origin
java -XX:+PrintFlagsFinal -version | grep -E "MaxHeapSize|UseG1GC|ParallelGCThreads"
#   size_t MaxHeapSize  = 4294967296    {product} {ergonomic}     <- chosen by the JVM
#   bool   UseG1GC      = true          {product} {ergonomic}
#   uintx  ParallelGCThreads = 8        {product} {default}

export JAVA_TOOL_OPTIONS="-Xmx2g -Dfile.encoding=UTF-8"    # any JVM on the machine, incl. mvn/gradle
export JDK_JAVA_OPTIONS="-XX:+UseZGC"                       # only the `java` launcher
java @prod.jvm.args -jar app.jar                            # one option per line in prod.jvm.args

java -XX:+UnlockDiagnosticVMOptions -XX:+PrintFlagsFinal -version | grep -i diagnostic | head
jcmd <pid> VM.flags                                         # what the RUNNING JVM uses
jcmd <pid> VM.set_flag HeapDumpOnOutOfMemoryError true      # manageable flags only
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Setting `JAVA_OPTS` and wondering why nothing changed. It's a convention for shell wrappers (Tomcat's `catalina.sh`, your own entrypoint script), not an environment variable the JVM understands. Use `JAVA_TOOL_OPTIONS` or `JDK_JAVA_OPTIONS`, or pass it explicitly in the launch command.

🛠️ **Q1179** PF-031 • Advanced

Give a production-ready JVM options baseline for a Spring Boot service on Kubernetes, with the reasoning for each flag.

⚡ **Quick answer**

A sensible baseline is small, explicit and based on the container: **percentage-based heap** (`MaxRAMPercentage`) so one image fits any pod size, **crash on OOM** so Kubernetes restarts the pod instead of leaving a zombie, a **heap dump on OOM** to a mounted volume, **bounded Metaspace**, an **explicit collector**, and **GC logging to a file with rotation**. Everything else (young sizing, tenuring, many GC flags) stays at defaults until measurements justify a change.

Add per-service decisions on top (for example `-XX:+UseZGC` for a strict latency SLO, `-XX:MaxDirectMemorySize` for Netty-heavy services), and keep the flags in **one place** (an image layer or a ConfigMap) so every service behaves the same. The reasoning for the 70% figure is the footprint maths in Q1161.

Terminal

```
# Dockerfile
ENV JAVA_TOOL_OPTIONS="\
 -XX:+UseG1GC \
 -XX:MaxRAMPercentage=70 -XX:InitialRAMPercentage=50 \
 -XX:MaxMetaspaceSize=256m \
 -XX:+ExitOnOutOfMemoryError \
 -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps \
 -XX:+UnlockDiagnosticVMOptions -XX:+DebugNonSafepoints \
 -Xlog:gc*:file=/logs/gc.log:time,uptime,level,tags:filecount=5,filesize=20m \
 -Djava.security.egd=file:/dev/urandom \
 -Duser.timezone=UTC"
ENTRYPOINT ["java", "-jar", "/app/app.jar"]

# Why each one
#  MaxRAMPercentage=70    heap = 70% of the pod limit; the rest covers metaspace, stacks, direct, GC
#  ExitOnOutOfMemoryError die fast and visibly; the orchestrator restarts a clean process
#  HeapDumpOnOOM + path   evidence for the post-mortem (mount an emptyDir/volume with room)
#  MaxMetaspaceSize       a class-loader leak becomes an error, not an OOMKill
#  DebugNonSafepoints     more accurate JFR/async-profiler stacks, no measurable cost
#  Xlog gc to a file      GC history survives the pod restart if /logs is a volume or shipped
```


© TechTribers. For personal use only. Do not redistribute.

Q1180 PF-032 • Advanced

How should you set Kubernetes requests and limits for a JVM workload (memory, CPU, QoS, throttling)?

⚡ Quick answer

Memory: the limit is a **hard cap**; exceeding it means `OOMKilled` (exit 137). Set **request = limit** for memory (no overcommit surprises) and size the heap as a **percentage of the limit** (60–75%). **CPU**: the request drives scheduling; a **limit** causes **CFS throttling**, which freezes all threads for the rest of the 100 ms period after the quota is spent, a classic cause of **p99 latency spikes** for JVMs (GC, JIT and request threads all burn the same quota). Many teams set a CPU **request without a limit** or a generous limit for latency-sensitive services.

If you do set a CPU limit, make it **at least 2 CPUs** for JVMs with concurrent GC, and align `ActiveProcessorCount` with it. Watch `container_memory_working_set_bytes` against the limit: that is what the OOM killer compares, not the heap metric. Startup needs more CPU than steady state (JIT, class loading); see the warm-up question (Q1193).

Terminal

```
# deployment.yaml (container section)
resources:
  requests: { cpu: "1500m", memory: "2Gi" }
  limits:   { memory: "2Gi" }                  # memory limit = request; no CPU limit (see below)
env:
  - name: JAVA_TOOL_OPTIONS
    value: "-XX:MaxRAMPercentage=70 -XX:+ExitOnOutOfMemoryError"
# heap = 0.7 x 2 GiB = ~1.4 GiB; ~0.6 GiB left for metaspace, stacks, direct buffers, GC structures

# Quality of Service: requests == limits for both cpu and memory -> Guaranteed (evicted last)
#                     requests < limits or only some set        -> Burstable
#                     nothing set                                -> BestEffort (evicted first)

# Is throttling hurting me?  (cgroup v2)  nr_throttled and throttled_usec must stay ~0
kubectl exec <pod> -- cat /sys/fs/cgroup/cpu.stat
# Prometheus: rate(container_cpu_cfs_throttled_periods_total[5m]) /
#   rate(container_cpu_cfs_periods_total[5m])
```

One 2 GiB pod: where the memory limit goes



memory: request = limit · limit exceeded = OOMKilled (exit 137), even with a healthy heap
CPU: a hard limit means CFS throttling: all threads freeze for the rest of the 100 ms period
→ p99 spikes. Prefer a CPU request without a limit, or a limit ≥ 2 CPUs with concurrent GC.

Check: cat /sys/fs/cgroup/cpu.stat (nr_throttled, throttled_usec) · container_memory_working_set_bytes

© TechTribers. For personal use only. Do not redistribute.

Q1181 PF-033

• Intermediate

Which system properties (-D) matter for a Java service: encoding, temp dir, IPv4, headless, JMX?

⚡ Quick answer

Useful ones: `file.encoding` (UTF-8 is the default since JDK 18; set it explicitly on older JVMs), `user.timezone` (avoid depending on the host zone), `java.io.tmpdir` (a writable, sized location in read-only containers), `java.net.preferIPv4Stack` (avoids slow or broken IPv6 lookups in some networks), `java.awt.headless=true` (image/PDF libraries on servers), and the **JMX remote** properties, which should be used carefully (authentication and TLS, or not at all).

Terminal

```
java -Dfile.encoding=UTF-8 -Dstdout.encoding=UTF-8 \
     -Duser.timezone=UTC -Duser.language=en -Duser.country=US \
     -Djava.io.tmpdir=/tmp/app \
     -Djava.net.preferIPv4Stack=true \
     -Djava.awt.headless=true \
     -jar app.jar

jcmd <pid> VM.system_properties | sort | head -40            # what is actually set

# Spring Boot: -D works, but properties and env vars are more idiomatic
#   java -jar app.jar --server.port=8081  |  SERVER_PORT=8081  |  -Dserver.port=8081

# JMX remote (only if you really need it; prefer actuator + metrics)
#   -Dcom.sun.management.jmxremote.port=9010 -Dcom.sun.management.jmxremote.authenticate=true
#   -Dcom.sun.management.jmxremote.ssl=true -Dcom.sun.management.jmxremote.access.file=...
```

⚠️ Interview trap

Opening JMX on a port with `authenticate=false` and `ssl=false` "temporarily". An unauthenticated JMX endpoint allows remote code execution; treat it like an open admin console.

Q1182 PF-034

• Intermediate

How do you configure TLS and truststores for the JVM (custom CA, javax.net.ssl, protocols)?

⚡ Quick answer

The JVM trusts the CAs in `$JAVA_HOME/lib/security/cacerts` (password `changeit`). For a **corporate or private CA**, either **import it into the JDK truststore** at image build time (`keytool -importcert -cacerts`) or point the JVM at your own truststore with `-Djavax.net.ssl.trustStore` (which **replaces** the defaults). Avoid passing passwords on the command line (visible in `ps`): use Spring Boot **SSL bundles** or environment/secret files. Restrict protocols with `jdk.tls.client.protocols`, and debug with `-Djavax.net.debug`.

© TechTribers. For personal use only. Do not redistribute.

`PKIX path building failed` always means "the server's certificate chain doesn't end in a CA this JVM trusts": missing intermediate, private CA not imported, or a TLS-inspecting proxy. Find which truststore the JVM actually used (`-Djavax.net.debug=ssl:trustmanager`) before changing anything.

Terminal

```
# Image build: add the corporate root CA to the JDK's own truststore
keytool -importcert -noprompt -trustcacerts -alias acme-root \
        -file /certs/acme-root-ca.pem -cacerts -storepass changeit

# Or a separate truststore (REPLACES the default CAs: include public CAs too if you need them)
java -Djavax.net.ssl.trustStore=/etc/tls/truststore.p12 \
     -Djavax.net.ssl.trustStoreType=PKCS12 -jar app.jar        # password via a secret, not argv

# Restrict protocols
java -Djdk.tls.client.protocols=TLSv1.3,TLSv1.2 -Dhttps.protocols=TLSv1.3,TLSv1.2 -jar app.jar

# Troubleshoot a handshake failure (very verbose: use briefly)
java -Djavax.net.debug=ssl:handshake:verbose -jar app.jar 2>&1 \
     | grep -i -E "alert|trust|certificate"

# Spring Boot 3.1+: SSL bundles keep key material in properties, not JVM flags
#   spring.ssl.bundle.pem.client.truststore.certificate=file:/etc/tls/ca.pem
```

Q1183 PF-035 • Advanced

How does JVM DNS caching work, and how do you configure it (networkaddress.cache.ttl, keep-alive, timeouts)?

⚡ Quick answer

The JVM caches DNS answers for **30 seconds** by default (when no security manager is installed); the security property `networkaddress.cache.ttl` (or the legacy system property `sun.net.inetaddr.ttl`) changes it, and `networkaddress.cache.negative.ttl` controls caching of failures. **With a SecurityManager the default is "cache forever"**, a legacy trap. More important in practice: **pooled HTTP and DB connections outlive DNS changes**, so a failover can be ignored for hours unless connections have a **maximum lifetime**. Always set **connect and read timeouts**.

Terminal

```
# java.security (or set programmatically before first lookup)
#   networkaddress.cache.ttl=30
#   networkaddress.cache.negative.ttl=5
java -Dsun.net.inetaddr.ttl=30 -jar app.jar                    # equivalent system property

# Programmatic (must run before any name lookup)
#   Security.setProperty("networkaddress.cache.ttl", "30");

# Connection pools: recycle connections so DNS/failover changes are picked up
#   spring.datasource.hikari.max-lifetime=1500000           # 25 min, below LB/proxy idle timeouts
```


© TechTribers. For personal use only. Do not redistribute.

```
#   HttpClient (Apache): .setConnectionTimeToLive(5, TimeUnit.MINUTES)
#   OkHttp: ConnectionPool + a dispatcher; Reactor Netty: maxLifeTime in ConnectionProvider

# Timeouts: no default is safe
#   HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(2))
#   request.timeout(Duration.ofSeconds(5))
#   JDBC: socketTimeout/connectTimeout; Redis, Kafka clients: request.timeout.ms
```

⚠️ **Interview trap**

Relying on DNS failover for a database or service behind a long-lived connection pool. The JVM re-resolves the name, but the pool keeps talking to the old IP until connections are recycled or fail.

Q1184 PF-036 • Intermediate

How do entropy and SecureRandom affect startup and performance (securerandom.source, egd)?

⚡ **Quick answer**

`new SecureRandom()` on Linux uses the **NativePRNG** backed by `/dev/urandom`, which never blocks. Blocking startup delays came from older kernels and VMs where `/dev/random` ran out of entropy and the JVM was configured to read it; on modern kernels (5.6+) it no longer blocks after initialisation. The old workaround `-Djava.security.egd=file:/dev/./urandom` still appears in many scripts and is harmless. `SecureRandom.getInstanceStrong()` may block by design: don't use it in request paths. Performance-wise, one shared `SecureRandom` can become a **contention point**; for non-security randomness use `ThreadLocalRandom` / `RandomGenerator`.

Java

```java
// Correct: security-sensitive values (tokens, keys, salts)
SecureRandom secureRandom = new SecureRandom();           // create once, thread-safe, reuse
byte[] token = new byte[32];
secureRandom.nextBytes(token);

// Avoid in request paths: can block waiting for entropy on some platforms
SecureRandom strong = SecureRandom.getInstanceStrong();

// Not security-sensitive (sampling, jitter, test data): faster and contention-free
int jitterMs = ThreadLocalRandom.current().nextInt(50, 200);
RandomGenerator fast = RandomGenerator.of("L64X128MixRandom");     // Java 17+ algorithms

// Slow startup on an old VM? check kernel entropy and the configured source
//   cat /proc/sys/kernel/random/entropy_avail
//   -Djava.security.egd=file:/dev/urandom     (java.security: securerandom.source)
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> If `jstack` at startup shows threads in `SecureRandom.getSeed` or `NativePRNG`, entropy is the bottleneck. On containers running new kernels this is rare; verify before cargo-culting the flag.

Q1185 PF-037 • Intermediate

Which old JVM flags no longer work, and what replaced them (PermSize, CMS, biased locking, PrintGC)?

> ⚡ **Quick answer**
> Migrating Java 8 launch scripts to 17/21/25, flags fall into three groups: **ignored with a warning**, **removed (the JVM refuses to start)** and **replaced**. Unknown `-XX` options make the JVM **exit** unless `-XX:+IgnoreUnrecognizedVMOptions` is set, so a stale flag can take a service down at deploy time. Clean the list rather than suppressing the error.

Terminal

```
# Old flag (Java 8)                                  Today
-XX:PermSize / -XX:MaxPermSize                       -XX:MetaspaceSize / -XX:MaxMetaspaceSize
-XX:+UseConcMarkSweepGC (CMS) and CMS flags          removed in JDK 14: use G1, ZGC, Shenandoah
-XX:+UseParNewGC, -XX:+CMSParallelRemarkEnabled      removed with CMS
-XX:+UseBiasedLocking / -XX:-UseBiasedLocking        removed in JDK 18: delete the flag
-XX:+AggressiveOpts                                  removed
-XX:+PrintGCDetails -XX:+PrintGCTimeStamps -Xloggc:  -Xlog:gc*:file=gc.log:time,uptime,level,tags
-Xverify:none / -noverify                            deprecated; leave class verification on
--illegal-access=permit                              removed in 17: use --add-opens per package
-XX:+UseStringDeduplication                          still valid (G1, ZGC, Shenandoah)
-Djava.endorsed.dirs / -Djava.ext.dirs               removed (JDK 9): module path or classpath

# Find what the JVM rejects, before deploying
java -XX:+PrintFlagsFinal -version 2>&1 | head      # prints "Unrecognized VM option 'X'" and exits
JAVA_TOOL_OPTIONS="$OLD_FLAGS" java -version   # test the exact option string on the target JDK
```

> ⚠️ **Interview trap**
> Adding `-XX:+IgnoreUnrecognizedVMOptions` to "make the warning go away". It hides flags that silently no longer do anything (your tuning is gone) and masks typos in flags that matter.

© TechTribers. For personal use only. Do not redistribute.

Q1186 PF-038

• Intermediate

What do you need to know about default charset, time zone and locale on the JVM?

⚡ Quick answer

Since **JDK 18 (JEP 400)** the default charset is **UTF-8** on every platform, so `new String(bytes)` and `FileReader` no longer depend on the OS; on older JVMs `file.encoding` followed the host, causing the classic works-on-my-machine encoding bugs. The default **time zone** still comes from the host (or `TZ` / `-Duser.timezone`), and the default **locale** from the OS: both change formatting and parsing (`String.format`, `toUpperCase`, `NumberFormat`). Be explicit in code, and run production JVMs in **UTC**.

Time-zone rules change (governments adjust DST): keep the JDK **patched** so `tzdata` is current, or update it with the tzupdater tool; this is a real production cause of off-by-an-hour bugs.

Java

```
// Be explicit: never depend on defaults
String text = new String(bytes, StandardCharsets.UTF_8);
Files.readString(path, StandardCharsets.UTF_8);
try (Writer w = Files.newBufferedWriter(path, StandardCharsets.UTF_8)) { w.write(text); }

// Locale: case conversion and formatting
"TITLE".toLowerCase(Locale.ROOT);                  // Turkish default locale would yield a dotless i
String.format(Locale.ROOT, "%.2f", amount);        // not 12,50 in a German default locale

// Time: store instants, convert at the edge, never use the JVM default zone for business rules
Instant now = Instant.now();
ZonedDateTime local = now.atZone(ZoneId.of("Asia/Kolkata"));

// What does this JVM use?
System.out.println(Charset.defaultCharset() + " " + System.getProperty("native.encoding")
        + " " + ZoneId.systemDefault() + " " + Locale.getDefault());
// -Duser.timezone=UTC -Duser.language=en -Duser.country=US pin them for the whole process
```

Q1187 PF-039

• Intermediate

What are --add-opens, --add-exports, --enable-preview and --enable-native-access, and where do you set them?

⚡ Quick answer

Since **JDK 17** the JDK's internals are **strongly encapsulated**: reflective access to `java.base` internals fails with `InaccessibleObjectException` unless you open the package: `--add-opens java.base/java.lang=ALL-UNNAMED` (deep reflection) or `--add-exports` (compile/link access to public types in non-exported packages). `--enable-preview` turns on preview language/API features (the **same JDK version** must compile and run the class). `--enable-native-access` acknowledges use of JNI and the Foreign Function & Memory API, which newer JDKs restrict or warn about.

© TechTribers. For personal use only. Do not redistribute.

Newer JDKs also warn when libraries use `sun.misc.Unsafe` memory-access methods, ahead of their removal (JEP 498): upgrade the libraries (Netty, Kryo, Hadoop clients and others have been releasing fixes) rather than adding flags.

Terminal

```
# Frameworks that reflect into JDK internals (older Hibernate/Spring libs, serialisation libs)
java --add-opens java.base/java.lang=ALL-UNNAMED \
     --add-opens java.base/java.util=ALL-UNNAMED \
     -jar app.jar

# Put them where the build tools see them too
export JDK_JAVA_OPTIONS="--add-opens java.base/java.lang=ALL-UNNAMED"
#   Maven Surefire:  <argLine>--add-opens java.base/java.lang=ALL-UNNAMED @{argLine}</argLine>
#   Gradle test:     jvmArgs("--add-opens", "java.base/java.lang=ALL-UNNAMED")

# Preview features: compile AND run with the same JDK and flag
javac --release 25 --enable-preview Main.java && java --enable-preview Main

# Native access (FFM API, JNI): grant it to the modules/jars that need it
java --enable-native-access=ALL-UNNAMED -jar app.jar

# Find which libraries need opens: run with the flag logging reflection failures
java -Djdk.module.illegalAccess.silent=false -jar app.jar
```

Q1188 PF-040 • Intermediate

How should you launch the JVM in Docker (exec form, PID 1, signals, JAVA_OPTS, graceful shutdown)?

⚡ **Quick answer**

Use the **exec form** `ENTRYPOINT ["java", "-jar", "app.jar"]` so the JVM is **PID 1** and receives `SIGTERM` directly; the shell form wraps it in `/bin/sh -c`, which may **not forward signals**, so the app is killed after the grace period instead of shutting down. Exec form doesn't expand `$JAVA_OPTS`, so pass flags with `JAVA_TOOL_OPTIONS` or call `exec java $JAVA_OPTS ...` from a shell. Enable **graceful shutdown** in Spring Boot and align it with Kubernetes' termination grace period.

Terminal

```
# Good: JVM is PID 1, gets SIGTERM, flags come from the environment
FROM eclipse-temurin:25-jre
COPY app.jar /app/app.jar
ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=70 -XX:+ExitOnOutOfMemoryError"
ENTRYPOINT ["java", "-jar", "/app/app.jar"]

# If you need shell expansion, make the shell exec the JVM (replace the shell process)
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/app.jar"]

# Bad: shell form: /bin/sh is PID 1 and may swallow SIGTERM
ENTRYPOINT java $JAVA_OPTS -jar /app/app.jar
```


© TechTribers. For personal use only. Do not redistribute.

```
# Graceful shutdown (Spring Boot)
#   server.shutdown=graceful
#   spring.lifecycle.timeout-per-shutdown-phase=25s
# Kubernetes: terminationGracePeriodSeconds: 40 (> 25 s), plus a preStop sleep (5 s) so the
# endpoint is removed from the load balancer before the app stops accepting connections
#   lifecycle: { preStop: { exec: { command: ["sleep", "5"] } } }
```

💡 **Pro tip**

If the JVM stays PID 1 and you fork child processes, use `docker run --init` (or `tini`) so zombie processes are reaped. Check that shutdown really is graceful: `kubectl delete pod` and look for "Commencing graceful shutdown" in the logs.

Section 05

JIT, Startup Time and Warm-up

Tiered compilation and the code cache, patterns that defeat inlining, faster startup with AppCDS and the AOT cache, CRaC and native image trade-offs, warming up pods before they take traffic, and a Spring Boot startup checklist.

Questions 1189–1194 · 6 questions



⚙️ **Q1189** PF-041 • Intermediate

How does tiered compilation work, and what is the code cache (ReservedCodeCacheSize)?

⚡ **Quick answer**

HotSpot starts in the **interpreter (tier 0)**, compiles hot methods with **C1** (tiers 1–3, fast to compile, with profiling at tier 3), and the hottest code with **C2** (tier 4, aggressive optimisation using the collected profile). Compiled code lives in the **code cache** (default about **240 MB** with tiered compilation). If it **fills up**, the JVM logs `CodeCache is full. Compiler has been disabled` and the application keeps running on **interpreted or old code**: a sudden, severe, silent slowdown.

Alert on **code cache usage** (`jvm.memory.used{id="CodeHeap ..."}`) or on the log message. It's rare for ordinary services but common in applications that **generate lots of code** (scripting engines, heavy proxy use, huge monoliths).

Terminal

```
java -XX:+PrintCompilation -jar app.jar | head
jcmd <pid> Compiler.codecache                          # size, used, free per segment
```


© TechTribers. For personal use only. Do not redistribute.

```
#   CodeHeap 'non-profiled nmethods': size=120032Kb used=15392Kb
#   CodeHeap 'profiled nmethods':     size=120028Kb used=61144Kb
jcmd <pid> Compiler.queue                              # methods waiting to be compiled

# Bigger cache if you see "CodeCache is full" (large apps, many generated classes/proxies)
java -XX:ReservedCodeCacheSize=512m -jar app.jar

# Trade peak speed for startup/footprint (C1 only): CLIs, dev mode, tiny containers
java -XX:TieredStopAtLevel=1 -jar app.jar

# Compiler threads scale with CPU count; check them (and CPU throttling at startup)
java -XX:+PrintFlagsFinal -version | grep CICompilerCount
```

Q1190 PF-042

• Advanced

What defeats JIT inlining and makes hot code slow (megamorphic calls, huge methods, exceptions, reflection)?

⚡ **Quick answer**

The JIT's biggest win is **inlining**, which exposes further optimisations (escape analysis, constant folding). It stops when: a **call site is megamorphic** (more than two receiver types seen: it falls back to a virtual or interface dispatch), a method is **too big** (hot methods above ~325 bytecodes are not inlined; methods over **8,000 bytecodes are never compiled**), the **call chain is too deep**, or the code uses **reflection and method handles that aren't constant**. Deoptimisation storms occur when profile assumptions break (a new subtype shows up late).

Terminal

```
# See what the JIT decides (diagnostic flags)
java -XX:+UnlockDiagnosticVMOptions -XX:+PrintInlining -XX:+PrintCompilation -jar app.jar 2>&1 \
  | grep -E "too big|megamorphic|callee is too large"
#   @ 14   com.acme.PriceStrategy::price (62 bytes)   virtual call              <- not inlined:
#   megamorphic
#   @ 22   com.acme.Report::render (412 bytes)        hot method too big

# Typical fixes (always confirm with JMH or a profile first)
#  1. Keep hot methods small; split cold branches (error handling, logging) into separate methods
#  2. Megamorphic interface in an inner loop: dispatch once outside the loop, or use a
#   switch/sealed types
#  3. Avoid reflection in hot paths: cache MethodHandle in a static final, or use generated
#   accessors
#  4. Don't create exceptions for control flow (stack trace capture defeats optimisation)
#  5. Huge generated methods (parsers, big switch tables): over 8000 bytecodes stays interpreted
java -XX:-DontCompileHugeMethods ...                # last resort: allow compiling huge methods
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Most application code never needs this level of analysis. Reach for `PrintInlining` only after a profile shows a **hot method that should be fast but isn't**, and verify any rewrite with JMH.

🚀 **Q1191** PF-043 • Advanced

How do you cut JVM startup time (AppCDS, JDK AOT cache, lazy initialisation)?

> ⚡ **Quick answer**
> Startup is dominated by **class loading and linking** and by the framework's own initialisation. **Class Data Sharing (CDS)** maps pre-parsed class metadata from an archive: the JDK ships one for its own classes, and **AppCDS** adds your application's classes (`-XX:ArchiveClassesAtExit` , or `-XX:+AutoCreateSharedArchive` since JDK 19). **JDK 24** added an **AOT cache** (JEP 483) that also stores loaded and linked classes, and **JDK 25** simplifies creating it (a one-step training run) and adds method profiles so warm-up is shorter. Typical gains: **20–50%** off startup, more for big Spring apps.

The archive is valid only for the **same JDK build and the same classpath**: regenerate it in CI whenever either changes, and keep a fallback (JVMs ignore a mismatched archive and just start normally). Verify the gain with real start-to-ready timings, not just class counts.

Terminal

```
# AppCDS (JDK 19+): the archive is created automatically on exit and reused on the next start
java -XX:+AutoCreateSharedArchive -XX:SharedArchiveFile=/tmp/app.jsa -jar app.jar

# Spring Boot: train with a startup-only run, then use the archive
java -XX:ArchiveClassesAtExit=application.jsa -Dspring.context.exit=onRefresh -jar app.jar
java -XX:SharedArchiveFile=application.jsa -jar app.jar

# JDK 25 AOT cache (JEP 514 ergonomics): one training run records, a flag uses the result
java -XX:AOTCacheOutput=app.aot -jar app.jar          # run a representative workload, then exit
java -XX:AOTCache=app.aot -jar app.jar                # faster startup and warm-up

# JDK 24: the same in explicit steps (record, create, run)
#   java -XX:AOTMode=record -XX:AOTConfiguration=app.aotconf -jar app.jar
#   java -XX:AOTMode=create -XX:AOTConfiguration=app.aotconf -XX:AOTCache=app.aot -jar app.jar

# Build the archive in the Docker image build, so every pod starts with it
#   RUN java -XX:ArchiveClassesAtExit=/app/app.jsa -Dspring.context.exit=onRefresh -jar
#   /app/app.jar
```


© TechTribers. For personal use only. Do not redistribute.

📸 Q1192 PF-044 • Advanced

What are CRaC and checkpoint/restore, and how do they compare with the AOT cache and GraalVM native image?

⚡ Quick answer

CRaC (Coordinated Restore at Checkpoint) snapshots a **fully warmed JVM process** to disk and **restores it in tens of milliseconds**, including JIT-compiled code, which removes both startup and warm-up cost. It needs a **CRaC-enabled JDK** (for example Azul Zulu or BellSoft Liberica builds) and application cooperation: sockets and files must be **closed before the checkpoint and reopened after** (Spring Boot 3.2+ has CRaC lifecycle support). **AOT cache/CDS** keeps the normal JVM and trims startup; **native image** compiles ahead of time for the fastest start and lowest memory, at the cost of reduced dynamism and peak throughput.

Native image is covered in more depth in Q315. In an interview, frame it as a **decision on workload shape**: not "which is best" but "what are we optimising: cold-start latency, density, or steady-state throughput?"

Terminal

```
# CRaC with a CRaC-enabled JDK (concept)
java -XX:CRaCCheckpointTo=/checkpoint -jar app.jar &          # run and warm up
jcmd <pid> JDK.checkpoint                                      # snapshot the warmed process
java -XX:CRaCRestoreFrom=/checkpoint                           # restore: tens of ms, already warm

# Choosing (rule of thumb)
#   Long-running services, steady load       -> plain JVM (+ AppCDS/AOT cache): best peak
#   performance
#   Scale-to-zero, serverless, many pods     -> CRaC or native image: fastest start
#   Strict memory/density targets            -> native image
#   Heavy reflection, dynamic proxies, agents -> stay on the JVM or invest in reachability metadata

# Trade-offs to state
#   CRaC:   same JIT peak speed, but the checkpoint image is large and tied to the platform;
#           secrets/connections must be re-established after restore
#   Native: no JIT warm-up, small RSS, but build time, no runtime bytecode generation, closed
#   world
```


© TechTribers. For personal use only. Do not redistribute.

Q1193 PF-045 • Advanced

How do you warm up JVM pods in Kubernetes so users don't see slow first requests?

⚡ **Quick answer**

A fresh JVM is **slow for its first seconds to minutes** (cold JIT, empty caches, lazy initialisation), and a rolling deploy sends it **full traffic immediately**. Layers of defence: a `startupProbe` and **readiness probe** so traffic arrives only when the app is ready; a **warm-up phase** that exercises hot paths (synthetic requests before reporting ready); **gradual traffic ramp-up** (service-mesh or load-balancer *slow start*); **extra CPU at startup** (higher limits, or a startup CPU boost) so JIT isn't throttled; and **AppCDS/AOT cache or CRaC** to shorten the cold period.

Terminal

```
# Probes: not ready until warmed
startupProbe:
  httpGet: { path: /actuator/health/liveness, port: 8080 }
  failureThreshold: 30
  periodSeconds: 5
readinessProbe:
  httpGet: { path: /actuator/health/readiness, port: 8080 }
  periodSeconds: 5

# Rolling update that never drops capacity
strategy: { rollingUpdate: { maxUnavailable: 0, maxSurge: 1 } }
```

```
// Warm-up in the app: report READY only after exercising the hot paths
@Component
class Warmup implements ApplicationRunner {
    private final ApplicationAvailability availability;
    private final ApplicationEventPublisher events;
    public void run(ApplicationArguments args) {
        IntStream.range(0, 2_000).forEach(i -> orderService.priceQuote(sampleRequest(i)));
        AvailabilityChangeEvent.publish(events, this, ReadinessState.ACCEPTING_TRAFFIC);
    }
}
```

```
# Traffic ramp: Istio  `trafficPolicy.loadBalancer.warmupDurationSecs: 60` (slow start for new
#   endpoints)
# AWS ALB: target group `slow_start.duration_seconds=60`
```

⚠️ **Interview trap**

Warm-up requests that **hit the real database or send real emails**. Use read-only or sandbox paths, and make sure warming doesn't pollute metrics (tag or exclude synthetic traffic).

© TechTribers. For personal use only. Do not redistribute.

Q1194 PF-046

• Intermediate

Give a startup and memory checklist for a Spring Boot service.

⚡ Quick answer

Find the cost first: enable the startup endpoint to see which beans take time. Then, roughly by payoff: **trim auto-configuration and classpath scanning** (exclude unused starters, use Spring AOT processing), consider **lazy initialisation** (faster start, but first-request latency and late failures), **disable what you don't use** (open-in-view, banner, JMX), **right-size pools** (Tomcat threads, Hikari, async executors), skip **JDBC metadata lookup** at boot, use **AppCDS/AOT cache**, and set container-aware JVM flags.

Terminal

```
# application.properties
# true: faster start, slower first requests, errors surface late
spring.main.lazy-initialization=false
spring.jpa.open-in-view=false
spring.main.banner-mode=off
spring.autoconfigure.exclude=\
  # only if you truly use none
  org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
server.tomcat.threads.max=100                  # default 200: fewer idle stacks if load allows
spring.datasource.hikari.maximum-pool-size=10
# Hibernate: skip the metadata round trip at startup (set the dialect explicitly)
spring.jpa.properties.hibernate.boot.allow_jdbc_metadata_access=false

# Where does startup time go?  Record and read the bean timeline
#   SpringApplication app = new SpringApplication(App.class);
#   app.setApplicationStartup(new BufferingApplicationStartup(4096));
#   management.endpoints.web.exposure.include=startup      ->  GET /actuator/startup

# Build-time work instead of run-time scanning
#   mvn -Pnative spring-boot:process-aot       (AOT processing for JVM or native)
#   java -XX:SharedArchiveFile=application.jsa -jar app.jar   (AppCDS, previous questions)
```

💡 Pro tip

Measure **time to ready** (container start to first successful readiness probe) rather than the log line "Started App in 6.2 seconds": the latter excludes JVM boot and early class loading.

© TechTribers. For personal use only. Do not redistribute.

Section 06

Code-Level Performance

Presizing collections and avoiding rehashing, streams versus loops and parallel stream traps, the real cost of synchronization, LongAdder and false sharing, thread pool sizing and virtual-thread scheduler settings, Caffeine caches, Jackson and JSON costs, I/O patterns, logging overhead and regex pitfalls.

Questions 1195–1203 · 9 questions



Q1195 PF-047

• Intermediate

How does collection resizing cost performance, and how do you presize ArrayList, HashMap and StringBuilder?

⚡ Quick answer

`ArrayList` grows by ~1.5× and **copies the array** each time; `HashMap` starts at 16 buckets and **doubles and rehashes** when 75% full; `StringBuilder` starts at 16 chars and doubles. Growing to a million elements triggers a couple of dozen copies and a lot of garbage. If you **know or can estimate the size, pass it**. `HashMap.newHashMap(n)` (Java 19) computes the right capacity for `n` entries, unlike `new HashMap<>(n)`, which is too small by the load factor.

Java

```
List<Order> orders = new ArrayList<>(expected);              // one allocation, no copies
Map<String, Order> byId = HashMap.newHashMap(expected);        // Java 19+: capacity for n entries
Set<String> seen = HashSet.newHashSet(expected);
StringBuilder sb = new StringBuilder(estimatedLength);

// Pre-19: new HashMap<>(expected) is NOT enough: capacity must be expected / 0.75 + 1
Map<String, Order> old = new HashMap<>((int) (expected / 0.75f) + 1);

// Streams: toList() sizes itself from a SIZED source; collect(toList()) may grow repeatedly
List<String> names = orders.stream().map(Order::name).toList();

// Right structure matters more than presizing
Deque<Task> queue = new ArrayDeque<>();                      // not LinkedList: no node allocations
Map<Status, Integer> counts = new EnumMap<>(Status.class);     // array-backed, no hashing
// Immutable and small: List.of / Map.of avoid defensive copies and hashing overhead
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Over-presizing: `new ArrayList<>(10_000_000)` "just in case" allocates 40–80 MB immediately and, for short-lived lists, creates the very humongous allocations you wanted to avoid (Q1170). Presize from real expected sizes.

Q1196 PF-048 • Intermediate

When are streams slower than loops, and when does parallel() help or hurt?

⚡ **Quick answer**

After JIT warm-up, **simple sequential streams** are close to loops; they get slower with **boxing** (`Stream<Integer>`), **captured-lambda allocations**, **short pipelines on tiny collections**, and complex `collect` pipelines in hot paths. `parallel()` helps only for **CPU-bound, stateless, cheaply splittable sources** (arrays, `ArrayList` , ranges), because it uses the shared **common ForkJoinPool** (cores - 1 threads) and has coordination overhead. It hurts with **blocking I/O**, small data, `LinkedList` / `iterate` / `Stream.of` sources, ordering requirements, and when other code shares the common pool.

Decision rule: write the **clearest** code first, measure with JMH or a profile, and only then specialise. A parallel stream in a request handler of a busy server competes with other requests for the same CPUs, so it rarely makes the **system** faster, only that one request.

Java

```
// Hot path: avoid boxing with primitive streams (or a plain loop)
long total = orders.stream().mapToLong(Order::amountCents).sum();       // no Long objects

// Parallel: CPU-bound, large, stateless, splittable source
long primes = IntStream.rangeClosed(2, 5_000_000).parallel().filter(Primes::isPrime).count();

// Parallel is WRONG here: blocking I/O in the common pool starves everything else using it
orders.parallelStream().forEach(o -> httpClient.send(toRequest(o), BodyHandlers.discarding()));
// Use an explicit executor (or virtual threads) for I/O fan-out
try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
    orders.forEach(o -> exec.submit(
            () -> httpClient.send(toRequest(o), BodyHandlers.discarding())));
}

// Dedicated pool when you really want parallel streams in a server
ForkJoinPool pool = new ForkJoinPool(4);
long result = pool.submit(() -> data.parallelStream().mapToLong(Heavy::compute).sum()).get();
```


© TechTribers. For personal use only. Do not redistribute.

🔐 **Q1197** PF-049 • Advanced

What does synchronization really cost, and when do LongAdder, StampedLock and @Contended help?

⚡ **Quick answer**

An **uncontended** lock is cheap (tens of nanoseconds). Cost explodes under **contention**: threads park and wake (context switches, microseconds), and cache lines bounce between cores. Tools: `LongAdder` (striped counters: far better than `AtomicLong` under many writers), `ConcurrentHashMap` (fine-grained locking; don't do I/O inside `compute`), `StampedLock` or `ReentrantReadWriteLock` for **read-heavy** data, **immutable snapshots** with a volatile reference, and **false-sharing** avoidance: independent hot variables on the **same 64-byte cache line** slow each other down (`@Contended`).

Java

```
// Hot counter with many writer threads
LongAdder requests = new LongAdder();         // contention-free increments; sum() is slightly stale
requests.increment();
long snapshot = requests.sum();
// AtomicLong: a single CAS target; throughput collapses with many cores incrementing

// Read-mostly configuration: publish an immutable snapshot, no locks on the read path
private volatile PriceTable table = PriceTable.empty();
void reload(PriceTable fresh) { this.table = fresh; }          // atomic swap
BigDecimal price(String sku) { return table.lookup(sku); }

// Don't do slow work inside a map lock
cache.computeIfAbsent(key, k -> remoteService.load(k));     // holds a bin lock during the call: BAD
// Better: Caffeine's LoadingCache, or load outside and putIfAbsent

// False sharing: two counters written by different threads, same cache line
@jdk.internal.vm.annotation.Contended   // JDK-internal; user code needs -XX:-RestrictContended
class Counters { volatile long reads; volatile long writes; }
```

⚠️ **Interview trap**

Replacing `synchronized` with `ReentrantLock` "for performance". Modern JVMs optimise `synchronized` well; switch only for features (timeouts, fairness, interruptibility) or after measuring contention. Shrinking the critical section helps far more than changing the lock type.

© TechTribers. For personal use only. Do not redistribute.

Q1198 PF-050

• Advanced

How do you size thread pools (CPU-bound vs I/O-bound, queues, ForkJoin, virtual-thread scheduler)?

⚡ Quick answer

CPU-bound work: about **N cores** threads (more just adds context switches). **I/O-bound**: `N × (1 + wait time / compute time)`, validated with **Little's law** (Q1150) and a load test. Pools need a **bounded queue and a rejection policy**: an unbounded queue hides overload until memory runs out, and `ThreadPoolExecutor` only creates threads beyond the core size when the queue is **full**. Use **separate pools per dependency** (bulkheads). The **common ForkJoinPool** has cores - 1 threads and must not run blocking code. **Virtual threads** run on a small carrier pool (`jdk.virtualThreadScheduler.parallelism`, default = cores), so you limit concurrency to scarce resources with **semaphores**, not pool size.

Observe pools in production: **active threads, queue size, rejected tasks and task wait time** (Micrometer exposes `executor.*` metrics). A pool that is always at max with a growing queue is saturated; raising `max` just moves the saturation to the database or downstream.

Java

```java
// CPU-bound
int cpu = Runtime.getRuntime().availableProcessors();
ExecutorService compute = Executors.newFixedThreadPool(cpu);

// I/O-bound: 50 ms waiting per 5 ms of compute on 8 cores -> 8 x (1 + 10) = 88 threads
ThreadPoolExecutor io = new ThreadPoolExecutor(
        32, 88, 60, TimeUnit.SECONDS,
        new ArrayBlockingQueue<>(200),                    // bounded queue
        new ThreadFactoryBuilder().setNameFormat("pricing-%d").build(),
        new ThreadPoolExecutor.CallerRunsPolicy());       // back-pressure instead of dropping

// Virtual threads: unlimited threads, bounded access to the scarce resource
Semaphore dbPermits = new Semaphore(20);
try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
    exec.submit(() -> {
        dbPermits.acquire();
        try { return repo.find(id); } finally { dbPermits.release(); }
    });
}

// Spring: the default @Async executor settings are rarely what you want
//   spring.task.execution.pool.core-size=16   queue-capacity=500   max-size=64
// JVM flag for virtual thread carriers: -Djdk.virtualThreadScheduler.parallelism=8
```


© TechTribers. For personal use only. Do not redistribute.

Q1199 PF-051 • Intermediate

How do you use an in-process cache like Caffeine correctly (size, expiry, stampede, memory)?

⚡ **Quick answer**

Caffeine is a high-performance in-heap cache. Configure a **bound** (`maximumSize` , or `maximumWeight` with a weigher), **expiry** (`expireAfterWrite` for correctness, `refreshAfterWrite` to reload hot keys in the background), and enable `recordStats` to watch the **hit rate**. A `LoadingCache` loads **one value per key at a time**, which prevents **cache stampedes**. Remember the cache is **part of your live set**: it counts against heap and affects GC, and it is **per instance**, so entries differ across pods.

Java

```
LoadingCache<String, Customer> customers = Caffeine.newBuilder()
        .maximumSize(50_000)                          // bound it: unbounded = memory leak
        .expireAfterWrite(Duration.ofMinutes(10))     // hard staleness limit
        // reload asynchronously while serving the old value
        .refreshAfterWrite(Duration.ofMinutes(2))
        .recordStats()
        .build(id -> customerClient.fetch(id));       // single load per key at a time

Customer c = customers.get("C-1001");

// Negative caching for expensive misses (short TTL)
Cache<String, Optional<Product>> lookups = Caffeine.newBuilder()
        .maximumSize(10_000).expireAfterWrite(Duration.ofSeconds(30)).build();

// Weight by size when entries vary widely
Cache<String, byte[]> blobs = Caffeine.newBuilder()
        .maximumWeight(64L * 1024 * 1024).weigher((String k, byte[] v) -> v.length).build();

CacheStats s = customers.stats();                 // hitRate(), evictionCount(), loadFailureCount()
// Spring: spring.cache.type=caffeine, spring.cache.cache-names=customers,
//         spring.cache.caffeine.spec=maximumSize=50000,expireAfterWrite=10m
```

⚠️ **Interview trap**

Caching mutable objects and handing the same instance to every caller. One caller modifies it and every other request sees the change. Cache **immutable** values (records) or defensive copies.

© TechTribers. For personal use only. Do not redistribute.

Q1200 PF-052 • Intermediate

How do you reduce JSON serialization cost with Jackson (ObjectMapper reuse, streaming, Blackbird, payload size)?

⚡ **Quick answer**

Typical costs: **creating an `ObjectMapper` per call** (very expensive: it caches serializers and introspection), **converting through intermediate trees or maps** (`readValue` to `Map` , then to DTO), **reflection overhead**, and **huge payloads**. Fixes: **one shared, configured `ObjectMapper`** , bind **directly to records or DTOs**, use the **streaming API** (`JsonParser` / `JsonGenerator`) for very large documents, enable the **Blackbird** module (bytecode-light accessors, JDK 17+), send **fewer fields** (projections), and **compress** large responses. For high-volume internal traffic, a binary format (Protobuf, Avro) is often 3–10× smaller and faster.

Java

```
// Shared, thread-safe once configured
private static final ObjectMapper MAPPER = JsonMapper.builder()
        .addModule(new BlackbirdModule())                  // faster property access (JDK 17+)
        .addModule(new JavaTimeModule())
        .disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES)
        .build();

// Bind directly to the target type: no Map or JsonNode in between
OrderDto dto = MAPPER.readValue(bytes, OrderDto.class);             // from byte[] (not String)
ObjectReader reader = MAPPER.readerFor(OrderDto.class);             // reuse readers on hot paths

// Stream a huge array: constant memory instead of loading 500 MB into objects
try (JsonParser p = MAPPER.getFactory().createParser(inputStream)) {
    p.nextToken();                                                  // START_ARRAY
    while (p.nextToken() == JsonToken.START_OBJECT) {
        process(MAPPER.readValue(p, OrderDto.class));
    }
}

// Response size: only the fields the client needs; gzip for large payloads
//   server.compression.enabled=true  server.compression.min-response-size=2KB
```

💡 **Pro tip**

Profile before switching formats: in many services JSON parsing is 5–10% of CPU while database and downstream calls dominate. Fix the biggest slice first.

© TechTribers. For personal use only. Do not redistribute.

Q1201 PF-053

• Intermediate

Which I/O patterns matter for performance (buffering, NIO, zero-copy, memory mapping, HTTP client reuse)?

⚡ **Quick answer**

Buffer small reads and writes (unbuffered `FileInputStream` makes a system call per read); use `Files.newBufferedReader/Writer` or wrap streams. **Stream large data** instead of loading it into memory. For file-to-socket transfers, `FileChannel.transferTo` uses **zero-copy** (`sendfile`). **Memory-mapped files** (`FileChannel.map`) suit large random-access reads but have unmapping and page-fault latency caveats. **Reuse HTTP clients** (connection pools, HTTP/2 multiplexing): creating one per request throws away TLS sessions and connections.

Java

```java
// Buffered, streamed processing of a large file: constant memory
try (BufferedReader r = Files.newBufferedReader(path, StandardCharsets.UTF_8)) {
    r.lines().map(Parser::parse).forEach(sink::accept);
}

// Zero-copy file to socket
try (FileChannel file = FileChannel.open(path, StandardOpenOption.READ)) {
    long sent = 0, size = file.size();
    while (sent < size) sent += file.transferTo(sent, size - sent, socketChannel);
}

// Batch small writes: one write of 64 KB instead of 64 writes of 1 KB
try (BufferedOutputStream out = new BufferedOutputStream(Files.newOutputStream(file), 64 * 1024)) {
    for (byte[] chunk : chunks) out.write(chunk);
}

// One shared HTTP client: connection pooling, HTTP/2, keep-alive
private static final HttpClient HTTP = HttpClient.newBuilder()
        .version(HttpClient.Version.HTTP_2)
        .connectTimeout(Duration.ofSeconds(2))
        .build();                                                    // create ONCE, reuse
```

⚠️ **Interview trap**

Creating a new `HttpClient` , `RestTemplate` or `WebClient` per request. Each carries its own connection pool and threads; the result is a leak of sockets and threads plus a TLS handshake on every call.

© TechTribers. For personal use only. Do not redistribute.

📝 **Q1202** PF-054 • Intermediate

How much does logging cost, and how do you keep it from hurting latency (async appenders, parameterised messages)?

⚡ **Quick answer**

Logging does string formatting, stack-trace capture, encoding and **synchronous I/O on the request thread**. Rules: use **parameterised messages** (`log.debug("id={}", id)` , no concatenation), **guard expensive arguments** (`isDebugEnabled()` or a lambda supplier), avoid logging in **tight loops**, avoid **caller location** patterns (`%C %M %L` capture a stack trace per event), and write through an **asynchronous appender** with a bounded queue so a slow disk doesn't stall requests. Choose the overflow policy deliberately: drop low-level events, never block critical paths.

Count log volume as a **cost**: 5,000 requests/s at 10 lines each is 50,000 events/s, which costs CPU, disk or network and money in your log platform. Log business events at INFO, details at DEBUG (off by default), and sample or rate-limit noisy errors.

Terminal

```
// Parameterised: no string building when the level is off (but arguments are still evaluated!)
log.debug("price for {} is {}", sku, price);
// supplier: evaluated only if enabled (Log4j2/SLF4J 2 fluent)
log.debug("order dump: {}", () -> expensiveJson(order));
log.atDebug().addArgument(() -> expensiveJson(order)).log("order dump: {}");

# logback-spring.xml: async appender with a bounded queue
<appender name="ASYNC" class="ch.qos.logback.classic.AsyncAppender">
  <queueSize>8192</queueSize>
  <discardingThreshold>20</discardingThreshold>   <!-- drop low levels at 80% full -->
  <neverBlock>true</neverBlock>                <!-- drop instead of blocking the request thread -->
  <appender-ref ref="JSON_STDOUT"/>
</appender>

# Pattern without caller data: avoid %C %M %L %F in production
#   %d{ISO8601} %-5level [%thread] %logger{36} - %msg%n

# Log4j2: all-async loggers (LMAX Disruptor) for the lowest overhead
#   -Dlog4j2.contextSelector=org.apache.logging.log4j.core.async.AsyncLoggerContextSelector
```


© TechTribers. For personal use only. Do not redistribute.

Q1203 PF-055

• Intermediate

What are the regex performance pitfalls (recompiling patterns, catastrophic backtracking / ReDoS)?

⚡ **Quick answer**

Two classic problems. **Recompiling**: `String.matches`, `replaceAll` and `split` with a regex **compile the pattern on every call**; compile once into a `static final Pattern`. **Catastrophic backtracking** (ReDoS): nested quantifiers such as `(a+)+$` or ambiguous alternations make the matcher try exponentially many paths on near-miss input, pinning a CPU core for seconds or minutes on a **single malicious request**. Fix by writing unambiguous patterns, using **possessive quantifiers or atomic groups**, limiting input length, using a **linear-time engine** (RE2J), and preferring plain `indexOf` / `startsWith` / `split(char)` when no regex is needed.

Java

```
// Compile once
private static final Pattern ORDER_ID = Pattern.compile("^ORD-\\d{8}$");
boolean ok = ORDER_ID.matcher(input).matches();     // not input.matches("^ORD-\\d{8}$") in a loop

// Vulnerable: nested quantifier; "aaaaaaaaaaaaaaaaaaaaaaaaaaaaa!" takes exponential time
Pattern bad = Pattern.compile("^(a+)+$");

// Safer: no ambiguity, possessive quantifier, bounded input
Pattern safe = Pattern.compile("^a++$");
if (input.length() > 256) throw new IllegalArgumentException("too long");

// Plain-string operations beat regex for simple cases
if (line.startsWith("ERROR")) { ... }
String[] cols = line.split(",", -1);                // single-char split has a fast path (no regex)

// Hard guard on untrusted patterns/inputs: a timeout wrapper or RE2J (guaranteed linear time)
//   com.google.re2j.Pattern.compile("...").matcher(input).matches()
// Detect: a thread dump shows a request thread RUNNABLE in java.util.regex.Pattern$... for
//   seconds
```

⚠️ **Interview trap**

Validating user-supplied input (emails, URLs, free text) with a clever regex copied from the internet. It is often the single easiest way to take down a service with one request; keep patterns simple and inputs bounded.

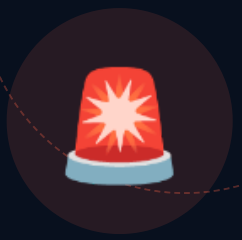
© TechTribers. For personal use only. Do not redistribute.

Section 07

Production Diagnostics and Runbooks

Memory-leak patterns in Spring services, diagnosing high CPU, a checklist for a service that is up but not responding, hunting periodic latency spikes, and the JVM metrics and alerts worth having.

Questions 1204–1208 · 5 questions



Q1204 PF-056

• Advanced

What are the most common memory-leak patterns in Java and Spring services, and how do you spot each?

⚡ Quick answer

A Java leak is **objects kept reachable that are no longer needed**. Common causes: **static or singleton maps/caches without eviction**, `ThreadLocal`**s on pooled threads** never removed, **unbounded queues and executors**, **listeners/callbacks never unregistered**, **high-cardinality metric tags** (each unique tag creates a new meter), **long-running persistence contexts** holding every loaded entity, `HttpSession`**/sessions piling up**, **class-loader leaks** (Metaspace), and **unclosed resources** (streams, connections). The signature is a **rising old-generation floor** after each GC.

Fix the **retention path**, not the symptom. In the heap dump's dominator tree or "path to GC roots" view, the chain from a GC root (a static field, a thread, a classloader) to the retained objects tells you exactly which reference to cut.

Java

```
// 1. Cache with no bound: grows with every distinct key
// use Caffeine with maximumSize
private static final Map<String, Report> REPORTS = new HashMap<>();

// 2. ThreadLocal on a pooled thread: value lives as long as the thread
private static final ThreadLocal<UserContext> CTX = new ThreadLocal<>();
try { CTX.set(ctx); handle(); } finally { CTX.remove(); }                // ALWAYS remove

// 3. High-cardinality tags create one Meter per value, forever
meterRegistry.counter("http.calls", "user", userId).increment();         // BAD: unbounded tags
meterRegistry.counter("http.calls", "route", route).increment();         // bounded tag values only

// 4. Unbounded executor queue / @Async: memory grows while the downstream is slow
Executors.newFixedThreadPool(8);                                  // LinkedBlockingQueue: unbounded!
// use a bounded ArrayBlockingQueue + rejection policy

// 5. Listener registered per request, never removed
eventBus.register(listener);   ...   eventBus.unregister(listener);
```


© TechTribers. For personal use only. Do not redistribute.

```
// How to spot: old-gen floor rising; histogram diff between two points in time
//   jcmd <pid> GC.class_histogram > h1.txt ; (30 min of traffic) ; jcmd <pid> GC.class_histogram
//   > h2.txt
//   diff the instance counts: the class that only grows is your suspect; then a heap dump
//   (Q335)
```

Q1205 PF-057

• Intermediate

How do you diagnose high CPU usage in a Java process?

⚡ **Quick answer**

Find the **hot thread**, then see **what it is doing**. `top -H -p <pid>` lists per-thread CPU; convert the busiest **thread id to hex** and match it to the `nid` in a thread dump, or use a profiler. Typical causes: an **infinite or very long loop**, **regex backtracking**, **excessive GC** (the busy threads are GC workers), **JIT compilation storms** at startup, **spinning/busy-wait** code, serialization of huge objects, or a runaway retry loop.

Terminal

```
top -H -p <pid>                       # per-thread CPU; note the TID of the hottest thread
printf "%x\n" <TID>                           # decimal thread id -> hex, e.g. 0x1f4a
jcmd <pid> Thread.print | grep -A15 "nid=0x1f4a"
#   "pool-4-thread-3" #41 runnable ... nid=0x1f4a
#       at java.util.regex.Pattern$Loop.match(Pattern.java:4796)          <- ReDoS / backtracking
#       at com.acme.Validator.isValid(Validator.java:58)

# Who is burning CPU: application threads or the JVM itself?
#   hot threads named "G1 Conc#0", "GC Thread#2"      -> GC: check the GC log and allocation rate
#   hot threads named "C2 CompilerThread0"            -> compilation storm (startup, deopt loops)
#   hot application threads                            -> profile it

asprof -d 30 -e cpu -f cpu.html <pid>        # CPU flame graph: the widest top frames are the answer
jcmd <pid> JFR.start duration=60s filename=/tmp/cpu.jfr settings=profile
```

💡 **Pro tip**

Take **two or three** thread dumps 5–10 seconds apart. A thread that appears in the same frame every time is stuck in that code; a thread that moves around is just busy.

© TechTribers. For personal use only. Do not redistribute.

Q1206 PF-058

• Advanced

The service is up but not responding. What do you check, in order?

⚡ Quick answer

Check **resource exhaustion** before blaming code. Take a **thread dump**: are all request threads waiting on the **same thing** (a downstream HTTP call, a **DB connection pool** with pending callers, one lock)? Is there a reported **deadlock**? Check **GC** (a death spiral looks like a hang: continuous Full GCs), **file descriptors** (`Too many open files`), **disk space**, **DNS** stalls, **CPU throttling**, and **ForkJoin common pool** starvation. Then look at what changed: deploy, config, traffic shape, a dependency's incident.

Most "hangs" turn out to be **pool exhaustion caused by one slow dependency**: threads pile up waiting, the pool fills, and healthy endpoints stop responding too. The fix is **timeouts, bulkheads and circuit breakers**, not bigger pools.

Terminal

```
# 1. What are the threads doing?
jcmd <pid> Thread.print > t.txt
grep -c "http-nio.*exec" t.txt;  grep "java.lang.Thread.State" t.txt | sort | uniq -c
grep -A8 "http-nio.*exec-1\"" t.txt                    # one representative request thread
#   at com.zaxxer.hikari.pool.HikariPool.getConnection     <- waiting for a DB connection
grep -i "Found .* deadlock" t.txt                      # the JVM reports Java-level deadlocks

# 2. GC death spiral?
jstat -gcutil <pid> 1000 5                             # FGC and GCT climbing, O near 100%

# 3. OS-level limits
ls /proc/<pid>/fd | wc -l;  ulimit -n;  cat /proc/<pid>/limits | grep "open files"
df -h /tmp /logs /dumps                                  # full disk blocks logging and dumps
cat /sys/fs/cgroup/cpu.stat | grep throttled             # CPU quota exhausted?

# 4. Pools and dependencies (actuator/Micrometer)
#   hikaricp_connections_pending > 0 for minutes, tomcat_threads_busy == max, downstream p99
#   timeouts
```

Q1207 PF-059

• Advanced

Latency spikes every few minutes: how do you find the cause?

⚡ Quick answer

Spikes that repeat on a schedule point to **periodic work**. **Correlate timestamps** of the spikes with every periodic source: **GC** (mixed collections, concurrent cycles), a scheduled `System.gc()` (RMI's distributed GC calls it **hourly** by default), **safepoint pauses** (log them), **CPU throttling** (cgroup quota), **connection pool recycling** (`maxLifetime`), **cache expiry stampedes**, cron jobs or batch work on the same node, DNS TTL expiry, and noisy neighbours.

© TechTribers. For personal use only. Do not redistribute.

Terminal

```
# Put every candidate on one timeline
java -Xlog:gc*,safepoint:file=/logs/gc.log:time,uptime,level,tags ...
grep -E "Pause|safepoint" /logs/gc.log | awk '{print $1, $NF}' | sort | tail -50    # pauses by time

# Hourly spike? RMI distributed GC calls System.gc()
#   GC(212) Pause Full (System.gc())                              <- the giveaway
# or avoid RMI/JMX remote
java -XX:+DisableExplicitGC -Dsun.rmi.dgc.server.gcInterval=3600000000 ...

# Safepoint time that is NOT GC (deopt, biased revocation on old JVMs, JFR, thread dumps)
#  Safepoint "ICBufferFull", Total: 612345 ns  ... a long Total = problem
java -Xlog:safepoint*=info ...

# CPU throttling bursts at the same minute?
#   rate(container_cpu_cfs_throttled_periods_total[1m])  vs latency graph
# Connection pool recycling: hikaricp max-lifetime default 30 min -> synchronized expiry
#   spring.datasource.hikari.max-lifetime=1500000   (HikariCP already adds jitter; check others)
# Cache stampede: many keys with the same TTL expire together -> add jitter / use
#   refreshAfterWrite
```

💡 **Pro tip**

Overlay latency percentiles with **GC pause time, CPU throttling, pool wait time and request rate** on one dashboard. The cause is usually visible within minutes once the graphs share a time axis.

📊 **Q1208** PF-060 • Intermediate

Which JVM metrics and alerts should every service have (Micrometer, Prometheus)?

⚡ **Quick answer**

Export **memory by area and heap-after-GC** (`jvm.memory.used/committed/max` , `jvm.gc.live.data.size`), **GC pauses and rate** (`jvm.gc.pause` , `jvm.gc.memory.promoted/allocated`), **threads** (`jvm.threads.live/peak/states`), **classes loaded**, **buffer pools** (`jvm.buffer.memory.used` direct), **CPU** (`process.cpu.usage`), **open file descriptors** against the limit, **pool metrics** (Hikari, Tomcat, executors) and the **container's working set and CPU throttling**. Alert on **symptoms and saturation**, not on every raw number.

Pair these with **service-level** alerts (error rate, latency percentiles, saturation of pools). A dashboard that shows **heap after GC, GC time share, thread count and pool queues** next to request latency answers most "is it the JVM?" questions at a glance, and the alerts above cover the failure modes this part described.

Terminal

```
# Spring Boot + Micrometer: JVM metrics are on by default with Actuator + Prometheus registry
#   management.endpoints.web.exposure.include=health,info,prometheus
#   management.metrics.distribution.percentiles-histogram.http.server.requests=true
```


© TechTribers. For personal use only. Do not redistribute.

```
# Prometheus alert rules (illustrative thresholds)
- alert: HeapAfterGcHigh
  expr: jvm_gc_live_data_size_bytes / jvm_gc_max_data_size_bytes > 0.8
  for: 15m
- alert: GcPauseP99High
  expr: histogram_quantile(0.99, sum(rate(jvm_gc_pause_seconds_bucket[5m])) by (le, pod)) > 0.2
  for: 10m
- alert: GcOverhead
  expr: rate(jvm_gc_pause_seconds_sum[5m]) > 0.10                # >10% of time paused
- alert: ThreadsGrowing
  expr: deriv(jvm_threads_live_threads[30m]) > 0.5
- alert: FileDescriptorsNearLimit
  expr: process_files_open_files / process_files_max_files > 0.8
- alert: PodMemoryNearLimit
  expr: container_memory_working_set_bytes / container_spec_memory_limit_bytes > 0.9
- alert: CpuThrottled
  expr: rate(container_cpu_cfs_throttled_periods_total[5m])
        / rate(container_cpu_cfs_periods_total[5m]) > 0.1
```


© TechTribers. For personal use only. Do not redistribute.

Series progress

1208 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 24 starts at Q1209.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49
Part 4	Q171–Q204	34
Part 5	Q205–Q254	50
Part 6	Q255–Q304	50
Part 7	Q305–Q339	35
Part 8	Q340–Q372	33
Part 9	Q373–Q397	25
Part 10	Q398–Q427	30
Part 11	Q428–Q479	52
Part 12	Q480–Q529	50
Part 13	Q530–Q579	50
Part 14	Q580–Q679	100
Part 15	Q680–Q739	60
Part 16	Q740–Q799	60
Part 17	Q800–Q849	50
Part 18	Q850–Q924	75
Part 19	Q925–Q974	50
Part 20	Q975–Q1034	60
Part 21	Q1035–Q1086	52
Part 22	Q1087–Q1148	62
Part 23	Q1149–Q1208	60

© TechTribers. For personal use only. Do not redistribute.

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

© TechTribers. For personal use only. Do not redistribute.